Leveraging Reviewer Experience in Code Review Comment Generation

HONG YI LIN and PATANAMON THONGTANUNAM, The University of Melbourne, Melbourne, Australia
CHRISTOPH TREUDE, Singapore Management University, Singapore, Singapore
MICHAEL W. GODFREY, University of Waterloo, Waterloo, Ontario, Canada
CHUNHUA LIU and WACHIRAPHAN CHAROENWET, The University of Melbourne, Melbourne, Australia

Modern code review is a ubiquitous software quality assurance process aimed at identifying and resolving potential issues (e.g., functional, evolvability) within newly written code. Despite its effectiveness, the process demands large amounts of effort from the human reviewers involved. To help alleviate this workload, researchers have trained various deep learning-based language models to imitate human reviewers in providing natural language code reviews for submitted code. Formally, this automation task is known as code review comment generation. Prior work has demonstrated improvements in code review comment generation by leveraging machine learning techniques and neural models, such as transfer learning and the transformer architecture. However, the quality of the model-generated reviews remains sub-optimal due to the quality of the open-source code review data used in model training. This is in part due to the data obtained from open-source projects where code reviews are conducted in a public forum, and reviewers possess varying levels of software development experience, potentially affecting the quality of their feedback. To accommodate this variation, we propose a suite of experience-aware training methods that utilise the reviewers' past authoring and reviewing experiences as signals for review quality. Specifically, we propose experience-aware loss functions (ELF), which use the reviewers' authoring and reviewing ownership of a project as weights in the model's loss function. Through this method, experienced reviewers' code reviews yield larger influence over the model's behaviour. Compared to the SOTA model, ELF was able to generate higher quality reviews in terms of accuracy (e.g., +29% applicable comments), informativeness (e.g., +56% suggestions), and issue types discussed (e.g., +129% functional issues identified). The key contribution of this work is the demonstration of how traditional software engineering concepts such as reviewer experience can be integrated into the design of AI-based automated code review models.

CCS Concepts: • **Software and its engineering → Software creation and management**; • **Computing methodologies → Machine translation**; **Natural language generation**;

P. Thongtanunam was supported by the Australian Research Council's Discovery Early Career Researcher Award (DECRA) funding scheme (DE210101091).
Authors' Contact Information: Hong Yi Lin (corresponding author), The University of Melbourne, Melbourne, Australia; e-mail: holin2@student.unimelb.edu.au; Patanamon Thongtanunam, The University of Melbourne, Melbourne, Australia; e-mail: patanamon.t@unimelb.edu.au; Christoph Treude, Singapore Management University, Singapore, Singapore; e-mail: ctreude@smu.edu.sg; Michael W. Godfrey, University of Waterloo, Waterloo, Ontario, Canada; e-mail: migod@uwaterloo.ca; Chunhua Liu, The University of Melbourne, Melbourne, Australia; e-mail: chunhua@student.unimelb.edu.au; Wachiraphan Charoenwet, The University of Melbourne, Melbourne, Australia; e-mail: wcharoenwet@student.unimelb.edu.au.

ACM 1557-7392/2026/4-ART126
https://doi.org/10.1145/3762183

Additional Key Words and Phrases: Code Review, Review Comments, Neural Machine Translation, Natural Language Generation

ACM Reference format:
Hong Yi Lin, Patanamon Thongtanunam, Christoph Treude, Michael W. Godfrey, Chunhua Liu, and Wachiraphan Charoenwet. 2026. Leveraging Reviewer Experience in Code Review Comment Generation. *ACM Trans. Softw. Eng. Methodol.* 35, 5, Article 126 (April 2026), 34 pages.
https://doi.org/10.1145/3762183

1 Introduction

As a spiritual successor to the Fagan inspection, modern code review is a lightweight human-oriented software quality assurance process that can be found in most of today's collaborative software development environments [23, 70, 89]. The process requires developers who are not the code author to inspect a code change for a wide variety of problems, ranging from functional issues (e.g., logical flaws and resource misuse) to evolvability issues (e.g., poor variable naming and low readability) [58]. Whilst the review process helps improve the quality and maintainability of the software, practitioners often find the process both time-consuming [7] and mentally taxing [31]. When code reviews are not rigorously conducted, software files are still found to be defective [75], emphasising a need for automation.

To help alleviate this workload, researchers have attempted to automate the code review process via three sequential tasks: *code change quality estimation*, *code review comment generation* and *code refinement*. Respectively, these tasks replicate the developer's actions in deciding if a code change needs to be revised, describing what needs to be revised in a natural language review and finally revising the code according to that review. In this work, we focus on the *code review comment generation* task which has been shown to be the most challenging component of the three tasks [47]. This involves training deep language models [65, 88] to provide natural language reviews that identify a wide variety of issues within submitted code changes [45, 47, 84]. The state-of-the-art model, CodeReviewer [47], is a **Text-to-Text Transfer Transformer (T5)**-based transformer [65] that has been trained on a large-scale GitHub code review corpus. Although considerable effort has been invested into curating a diverse set of code reviews, little attention has been placed on exploring the variation in quality across the reviews themselves, i.e., all past studies including CodeReviewer have weighted the data uniformly during model training, allowing equal influence over model behaviour, despite potential differences in content quality.

Code reviews can be conducted by a wide range of reviewers with varying levels of expertise, which may lead to diversity in the quality of reviews. Prior studies found that reviewers' experience and expertise are often associated with the issue types identified in code reviews [40] and the level of usefulness [86]. Inexperienced reviewers (who have reviewed and authored few code changes) often focus on trivial issues like visual representation (e.g., Figure 1, Example 2), which are considered less useful by developers [40, 86], or express confusion and uncertainty (e.g., Figure 1, Example 3), an anti-pattern associated with a lack of experience [13]. In contrast, experienced reviewers are more likely to identify critical functional issues, such as the missing validation check in Figure 1, Example 1, which demonstrates the value of deeper code understanding [40].

As code reviews may reflect the reviewers' insights drawn from their prior software development and code review experiences, we hypothesise that the quality of generated reviews can be improved by aligning the language model with experienced reviewers' perspectives. To achieve this alignment, we propose a suite of experience-aware training methods that utilise the reviewers' past authoring and reviewing experiences as signals for review quality. Specifically, we propose a novel method called **Experience-Aware Loss Functions (ELF)**, which assigns a weight to the model's loss

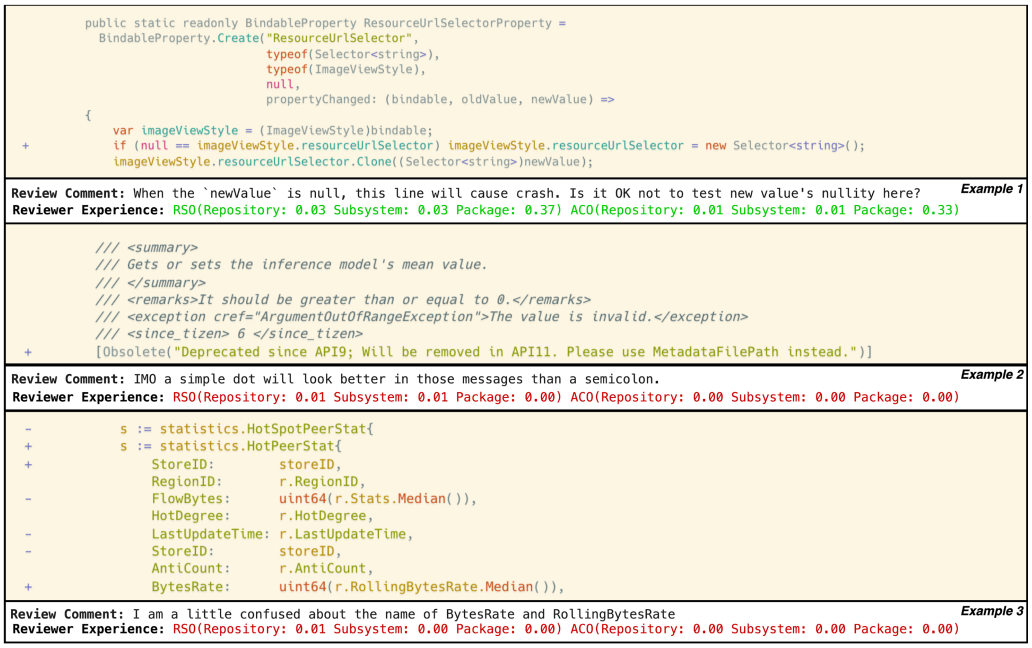

Fig. 1. Example of variation in code review content provided by reviewers with different levels of experience.

function that is proportional to the reviewer's experience in the project [9, 76]. In this way, comments made by experienced reviewers yield more influence over the model's behaviour.

Through both quantitative and qualitative evaluation, we found that ELF demonstrated stronger capability in generating high-quality reviews compared to past methods. In terms of accuracy, ELF achieved the highest increase over CodeReviewer in terms of BLEU-4 (+5%) and applicable comments generated (+29%). In terms of informativeness, ELF exhibited the highest increase in suggestions over CodeReviewer (+56%), whilst maintaining a similar improvement compared to ELF in terms of confused questions (−71%) and explanations generated (+125%). Regarding the types of generated reviews, we found that ELF demonstrated the highest increase over CodeReviewer in terms of functional faults detected (+129%) and evolvability issues identified (+21%). Amongst the different configurations of ELF, we found that considering both authoring and reviewing experience separately at the package level yielded the most improvement, however, the complementary nature of the different granularities should also not be overlooked.

This study extends our short paper [50], published at MSR 2024 (The International Conference on Mining Software Repositories), by introducing a new experience-aware technique along with deeper analysis to explore how different ways of estimating and weighing reviewers' experiences affect model-generated reviews. To recapitulate, the initial work [50] introduced experience-aware oversampling, which aims to improve code review comment generation models by oversampling experienced reviewer's comments during model training. This preliminary method identified experienced reviewers by utilising the traditional 5% ownership threshold rule [9, 76] from both code reviewing and authoring perspectives at the repository level. In this extension, our experimentation expands upon the initial work along two main vectors. First, we expand the calculation of the ownership metrics to consider finer granularities; that is, we consider subsystem and package level ownership in addition to repository level ownership that was originally considered in the short paper. The intention is to better reflect reviewers' specialised experiences within the project. Second,

we introduce ELF to replace the previously proposed experience-aware oversampling method; we do this for two reasons: (1) to mitigate the sensitivity to arbitrarily selected major ownership thresholds that may not generalise across projects, and (2) to circumvent the computationally costly search for optimal upsampling rates. We compare 12 new models derived from ELF in addition to the three previously proposed experience-aware oversampling strategies from the short paper (calculated at repository level only) and the original CodeReviewer model in terms of accuracy, informativeness and issue types discussed.

The main contributions of this article are:

— A suite of experience-aware training methods for improving code review comment generation.
— An analysis of emergent behaviours of code review comment generation models after experience-aware training.
— Two large-scale datasets containing commit and pull request histories for 826 of the top GitHub repositories.
— An augmented version of CodeReviewer's dataset that is tagged with six different ownership metrics.

2 Background and Related Work

In this section, we discuss related work from three main areas: automated code reviews (Section 2.1), review comment generation (Section 2.2) and experienced reviewers and code review quality (Section 2.3). We focus on these aspects as we seek to improve review comment generation by leveraging reviewer experience as a signal for identifying higher quality reviews during model training.

2.1 Automated Code Reviews

The recent success of deep learning-based language models in the field of natural language processing [17, 64, 65, 88] has inspired a plethora of research on their application to the software engineering domain [28, 52, 98]. This transition is seemingly natural, as both programming languages and natural languages are used to form sequential texts that conform to syntax and grammar, whilst expressing human logic in their semantics. As a result, many studies have demonstrated that language models can achieve promising levels of performance in real-world program understanding and generation tasks [98]—such as vulnerability detection [25], repair [24, 26, 100] and bug fixing [37, 38, 82]—suggesting the existence of exploitable naturalness properties [1, 36, 69] within human written software. Likewise, the field of automated code reviews also operates under the assumption that review comments are often repetitive and predictable, making the problem a suitable candidate for language modelling.

In its current form, automated code reviewing can be subdivided into a sequence of three tasks: code change quality estimation [35, 47], review comment generation [45, 47, 50, 84] and code refinement [44, 47–49, 62, 77, 81, 84, 85]. First, code change quality estimation requires the model to determine whether a code change submitted by a developer requires code review. This can be defined as a ($H_{pre} \rightarrow revise?$) binary classification task [47], where H_{pre} is the version of the code hunk submitted for review and $revise?$ is the binary decision output of whether the code change needs review. If it is determined that the code change needs review, review comment generation is subsequently performed. The review comment generation task (the subject of this study) requires the model to generate a natural language comment that can help guide the developer to improve the submitted piece of code, just as a human reviewer would. More formally, this can be formulated as a ($H_{pre} \rightarrow R_{nl}$) neural machine translation task [47], where R_{nl} is the natural language review comment. Finally, code refinement requires the language model to address the review comment R_{nl}, by revising H_{pre} to the final improved version of the code hunk H_{post}, ready to be merged into the code base. This last task can be formulated as a ($H_{pre}, R_{nl} \rightarrow H_{post}$) bimodal input translation problem [47].

Whilst it has been proven that underlying statistical properties of code reviews are learnable, publicly available state-of-the-art techniques such as LLaMA [51] and ChatGPT [32, 83] still struggle to perform well on the given test sets. In contrast, the code review models trained on closed-source software projects [23, 89] have reported high adoption rates. A likely reason being the data was collected from environments with experienced software engineers, strict code review cultures, and well-managed version control data, demonstrating that these code review models are performing well in practice when trained with high-quality examples. However, such data of closed-source projects are limited, resulting in the need for models to rely on data from open-source platforms such as GitHub, where the code review standards may vary widely. Our study focuses on alleviating this issue for the task of review comment generation, by leveraging reviewer experience as a proxy for identifying high-quality code reviews in the open source.

2.2 Review Comment Generation

Review comment generation (the subject of this study) represents the most challenging task in the attempt to automate code reviews, as the language model is required to infer the single ground truth comment from a vast space of potential solutions using only a small window of code. Tufano et al. [84] have demonstrated the potential of using the T5 [65] to generate code reviews, when pre-training the model with general code and natural language corpora related to the software engineering domain. Yet, compared to code refinement, the model showed far lower performance in terms of text matching metrics for review comment generation; however, upon their manual inspection, it was revealed that many generated comments were in fact either semantically equivalent to the ground truth or a valid alternative solution. In this study, we conduct manual evaluation to additionally account for these cases.

Whilst Tufano et al. [84] conducted pre-training on more general software engineering corpora, Li et al. [45] found success in jointly pre-training on code functions and their attached reviews. Simultaneously, CodeReviewer [47] showed significant performance leaps by training on code review-specific pre-training tasks as opposed to the generic masked language modelling technique used in prior works [45, 84]. This success was enabled by their large-scale multilingual GitHub code review dataset, which is now widely employed as the benchmark dataset [48, 50, 51, 72].

More recently, Lu et al. [51] discovered that generic LLMs such as LLaMA [80] were also capable of review comment generation by parameter-efficient fine-tuning on only 8.4 million parameters. Taking a different approach, Sghaier and Sahraoui [72] explored the effect of cross-task knowledge distillation, by jointly learning the successive tasks of review comment generation and code refinement together with two symbiotic models.

Whilst past research efforts have focused on improving review comment generation using various machine learning techniques, all of these approaches still assume the quality of comments across their open-source datasets are equivalent, i.e., their approaches allow all data to contribute uniformly to the model's learning process. This can be sub-optimal as the models will equally attempt to imitate the behaviour found in low-quality code reviews, as those of higher quality, which can reduce the potential usefulness of the resulting tool, as evidenced by our initial results [50]. In contrast, our current study proposes a novel method that forces more attention on potentially higher quality examples, such that they yield stronger influence over model behaviour, which may in turn increase the usefulness of the tool.

2.3 Experienced Reviewers and Code Review Quality

Detailed knowledge about program elements is usually retained by developers who frequently work with the code base [22]. In the context of software quality, code ownership has often been a determinant of software defect proneness, where ownership ratios are inversely related to defect

occurrences [9, 34, 76]. A study of implicated code has demonstrated that lower file level ownership tended to characterise the developers responsible for buggy lines [66], thus highlighting the importance of specialised knowledge. Concurrently, experienced developers are often assigned to fix complex bugs due to their expertise [12], which recursively deepens their knowledge of potential software quality issues. Given that code reviewers are a sub-population of developers, one would naturally intuit that their software development experience also has an impact on their effectiveness in reviewing code.

Thongtanunam et al. [76] studied the relationship between software defects and code ownership of reviewers at the module level, concluding that modules that were reviewed by developers who lack both code authoring and reviewing expertise were more likely to be defect-prone. Whilst studying the role of people and participation in code review quality, Kononenko et al. [41] also discovered corroborating evidence of the inverse relationship between reviewer experience and the presence of bugs, thus indicating that code review quality indeed varies depending on the individual who conducts it. A survey in the open-source Mozilla core project revealed that developers were in close to uniform agreement regarding reviewer experience being a factor that influences code review quality [40]. The developers reasoned that domain knowledge is crucial for properly evaluating a change, as superficial reviews arise from a lack of familiarity with the code base. From an industry perspective, findings from Microsoft demonstrated that having prior experience with a file under review has a noticeable effect on the usefulness of the reviews provided [10]. Whilst this line of research has studied the notion of useful comments coarsely defined by their ability to trigger a code change [10, 67], our study focuses on a more fine-grained view of review quality, scrutinising the variation in quality between change triggering comments.

Past work has proposed to automatically evaluate code review quality by directly classifying review comments; however, they often do not cover detailed technical concerns [10, 33, 67, 96], struggle to reach reliable accuracy [87], or are privately developed for specific software environments [96]. For example, Bosu et al. [10] and Rahman et al. [67] framed the problem as a binary classification task, using classical machine learning models to identify useful comments, i.e., change triggering comments. On the other hand, Yang et al. [96] proposed to assess comments based on linguistic attributes, i.e., emotion, question, evaluation, suggestion, using a separate binary classifier for each attribute. Although these types of classifications provide insight into communication dynamics and reviewer intent, they do not capture technical concerns of code changes under review, as both comments addressing critical issues regarding functional defects and trivial issues (e.g., visual representation) will trigger changes, evaluate code weaknesses and provide suggestions. Different to this line of work, other studies have investigated code review quality from the perspective of issue categories. Developers at Samsung [33] have expressed that reviews which identify defects, missing validations, performance optimisation opportunities, logical mistakes and so on are useful, whilst visual representation issues that can be identified by static analysis tools are not useful. Coinciding with these results, developers from the OpenDev project [86] rated reviews that discuss functional defects and validation issues as the most useful type of reviews, whilst those that address visual representation are rated as the least useful. Based on this spectrum of comment usefulness ratings, a reviewer's prior coding experience was found to have the largest positive impact on comment usefulness, whilst their authorship/reviewership of the specific file under review had no significant association. As the positive relationship between reviewer experience and code review quality has continuously resurfaced across different software development environments and has shown promising results in our initial study [50], we hypothesise that this attribute can directly serve as a general review quality signal for enhancing automated code review models in the absence of detailed and reliable review comment classifiers.

3 Experience-Aware Training Methods

Different to past approaches [45, 47], which have mainly focused on the effectiveness of deep learning-based language modelling techniques, our approach is the first to scrutinise the variation in quality amongst open-source code reviews. As prior studies [33, 40, 86] found that reviewer experience is positively correlated with review quality, we developed two experience-aware model training methods to improve the quality of model-generated code reviews. This section presents our proposed experience-aware training methods. In particular, we describe reviewer experience heuristics that are used to reflect a reviewer's software development experiences (Section 3.1), our previously proposed experience-aware oversampling method (Section 3.2) and our newly proposed ELF method (Section 3.3).

3.1 Reviewer Experience Heuristics

We measure reviewers' experience by calculating traditional code ownership metrics based on reviewers' past activity as a code reviewer [76] and as a code author [9]. We consider three granularity levels of software systems: repository, subsystem and package [97]. Ownership metrics at each granularity level represent different levels of knowledge coverage, where the repository level reflects general knowledge within a repository and the package level reflects knowledge of a particular component. We measure experience at different levels of granularity because some reviewers have ownership over specific components in the software system [79, 97], whilst other reviewers are maintainers that oversee the whole project at a macro level [18].

For authoring experience, **Authoring Code Ownership (ACO)** [9] represents the share of overall code contributions attributable to an individual. It represents a developer's experience and coverage on a piece of software gained from hands on coding. We formulate ACO as follows:

$$ACO(D, G) = \frac{\alpha(D, G)}{C(G)}, \quad G \in \{Repository, Subsystem, Package\}, \tag{1}$$

where $\alpha(D, G)$ is the number of commits in which reviewer D has contributed to the software at the targeted granularity $G \in \{Repository, Subsystem, Package\}$ and $C(G)$ is the total commits at that granularity. A higher ACO ratio indicates more experience as a code author for the targeted granularity in the system, and vice versa. Whilst other studies [3, 55] explore authorship at a line level fidelity, i.e., git blame, the scale of our study is orders of magnitude larger, rendering this calculation infeasible. A recent study also found that commit-based ownership shares a stronger relationship with software quality than line-based ownership [78]. Thus, we resort to commit level calculations, which is also widely considered as a reasonable approximation [9, 54, 76].

For a reviewing experience, **Review-Specific Ownership (RSO)** [76] represents the share of overall code reviews attributable to an individual. It represents a developer's experience and coverage of a part of software systems gained from reviewing code. We formulate RSO as follows:

$$RSO(D, G) = \frac{r(D, G)}{\rho(G)}, \quad G \in \{Repository, Subsystem, Package\}, \tag{2}$$

where $r(D, G)$ is the number of closed pull requests in which reviewer D has reviewed (i.e., commented at least once) for a given granularity $G \in \{Repository, Subsystem, Package\}$ and $\rho(G)$ is the total number of closed pull requests for that given granularity. A higher RSO ratio indicates more experience as a code reviewer for the chosen granularity in the system, and vice versa.

The three levels of granularity are determined as follows:

— *Repository level* is the most coarsely grained, where authoring and reviewing activities are considered in terms of the entire repository of the project under development. Specifically, all commits and reviewed pull requests mined from a repository are considered at this level. Ownership at the repository level represents general experience covering the entire project.
— *Subsystem level* is represented by the top level of file directories in the repository [97]. For example, the file ***arch***/*arm64/kernel/module.c* is considered to sit within the ***arch*** subsystem. A commit or reviewed pull request belongs to a subsystem if at least one of the changed files resides within that directory. Ownership at the subsystem level represents coverage over a particular top-level component of the system. Whilst we use the term 'subsystem' to refer to the top-level directories in the source code hierarchy, we recognise that this view may not match an experienced developer's mental model of the software architecture [42, 57, 74]. However, we consider that this model is a reasonable proxy: the source hierarchy is known to all developers of the project and hence has currency to them. In addition, the evaluation will be based on 826 GitHub repositories in our study; thus, devising a specialised architectural model for each repository is impractical and the models would be peculiar to our experiences rather than those of the project developers.
— *Package level* is represented by the immediate folder that contains the target file [97]. For example, the file ***arch/arm64/kernel***/*module.c* is considered to sit within the ***arch/arm64/kernel*** package. A commit or reviewed pull request belongs to a package if at least one of the changed files resides within that directory. Ownership at the package level represents coverage over a direct set of co-located files. Similarly, our measurement of a package is only an approximation; it does not reflect the ground-truth system architecture. The term 'package' here is defined in terms of the hierarchical view of file locations, not to be confused with programming language constructs such as Java packages or C++ namespaces.

3.2 Experience-Aware Oversampling

Experience-aware oversampling was designed to over represent experienced reviewers' code reviews during training, such that their perspectives yield more influence over the model's behaviour. We previously introduced the experience-aware oversampling method as a preliminary test of the concept that automated code review models could conform to the perspectives of experienced reviewers, thus improving the quality of generated reviews [50]. This method utilised the traditional 5% ownership threshold rule [9, 76] to target three specific sub-populations of the dataset, resulting in three separate models. The first group were major authors *MA* ($ACO \geq 5\%$), the second group were major reviewers *MR* ($RSO \geq 5\%$), and the last group were major reviewers *and* major authors *MRMA* ($ACO \geq 5\%$ *and* $RSO \geq 5\%$). Each model was trained by oversampling one of these groups with an oversampling rate of 400%. A sizable oversampling rate was chosen with the intention to elicit strong effects.

3.3 ELF

In this study, we introduce ELF, which uses the reviewers' assigned ownership ratios directly as loss function weights during training, such that their code reviews yield stronger influence over the model's behaviour. Specifically, we augment the original negative log-likelihood loss for review comment generation [47] with an additional experience embedded weight term ω [19, 46, 90–92]. This weight term dynamically accounts for the actual ownership values in their continuous form which both removes the need for the major ownership threshold assumption and the need for tuning an upsampling rate. Furthermore, utilising the ownership values in their original form retains continuous information and allows the model to capture the intricate differences between

the examples. We formulate the ELF as follows:

$$\mathcal{L}_{RCG} = \omega \sum_{t=1}^{k} -\log P(w_t|c, w_{<}t), \tag{3}$$

where c is the submitted code change, w_t is the current comment token, $w_{<}t$ are the comment tokens generated so far, and k is the sequence length. We formulate four weighting strategies to embed the experience values of ACO and RSO dimensions into the weight term ω.

- $\omega_{aco} = e^{1+aco}$ takes only authoring experience into consideration, speculating that reviewers who are prolific in coding provide higher quality code reviews. Intuitively, the higher the ACO of the reviewer who wrote the comment, the heavier the penalty (i.e., the loss value is amplified by the weight term ω) to the model for an incorrect prediction.
- $\omega_{rso} = e^{1+rso}$ takes only reviewing experience into consideration, speculating that reviewers who are prolific in reviewing pull requests provide higher quality code reviews. Intuitively, the higher the RSO of the reviewer behind the comment, the heavier the penalty to the model for an incorrect prediction.
- $\omega_{avg} = e^{1+\frac{rso+aco}{2}}$ considers both authoring and reviewing experience equally, speculating that reviewers who are prolific in both committing and reviewing code provide higher quality code reviews. Intuitively, the higher the combined ACO and RSO of the reviewer behind the comment, the heavier the penalty to the model for an incorrect prediction.
- $\omega_{max} = e^{1+max(rso,aco)}$ considers only the most representative experience type, speculating that reviewers who are prolific in either committing or reviewing code provide higher quality code reviews. Intuitively, the higher the reviewer's max ownership, regardless of experience type, the heavier the penalty to the model for an incorrect prediction.

Given that the model will be penalised differently based on the particular reviewer experience in focus, the weighted loss forces the model to align to their code review examples by affecting the direction of the gradient updates. Since the variation between ownership values is numerically small, we take an exponential to create stronger separation effects [19, 90, 93]. The four weights can be calculated at the three aforementioned granularities $G \in \{Repository, Subsystem, Package\}$, resulting in 12 different models. For example, the model trained with ω_{aco_Repo} weighted $\mathcal{L}_{RCG}$ conforms to reviewers who have high ACO at the repository level, whilst the model trained with ω_{avg_Pkg} weighted $\mathcal{L}_{RCG}$ aligns to reviewers with high coverage in terms of both authoring and reviewing ownership at the package level.

4 Experimental Setup

In this section, we propose our **Research Questions (RQs)** (Section 4.1), provide a study overview (Section 4.2), as well as describe our data preparation steps (Section 4.3), fine-tuning implementation (Section 4.4), evaluation metrics (Section 4.5) and manual evaluation process (Section 4.6).

4.1 RQs

In this work, we set out to investigate the effectiveness of experience-aware training methods on code review comment generation. We formulate three RQs to evaluate the model performance in three main aspects, i.e., accuracy, informativeness and issue types.

(RQ1) What is the impact of ELF on the accuracy of code review comment generation models?

Motivation. This RQ aims to evaluate the correctness of the generated comments against the ground truth, i.e., comments made by human reviewers. We measure the accuracy of the ELF models in

terms of textual similarity [61] and semantic equivalence [45, 84, 85] against the ground truth in the test set. Additionally, to capture the diverse nature of all potential code reviews, we also assess their general applicability [47, 84, 85] to the code change submission. The textual similarity metric (BLEU-4), will be automatically assessed, whilst both semantic equivalence and applicability are manually evaluated.

(RQ2) What is the impact of ELF on the informativeness of code review comment generation models?

Motivation. Informativeness is one of the important characteristics of high-quality reviews as perceived by developers [33, 40]. Typically, the informativeness of a code review refers to whether they identify an issue [10], prescribe a solution [33, 47] and provide a clear explanation for their rationale [40, 53, 68, 86]. The ability to achieve all three criteria may vary depending on the reviewer's experience. In this RQ, we set out to evaluate the quality of the code reviews generated by the ELF models in terms of feedback type, i.e., suggestions, concerns, confused questions and presence of explanation. Both feedback type and presence of explanation are metrics that involve manual evaluation.

(RQ3) What issue types are discussed in the ELF models' code reviews?

Motivation. Code reviews cover a wide variety of issues, including both functional and non-functional properties of the system [58]. Technical issues such as those related to defects and logic often require more in-depth knowledge of the system [40, 86] as opposed to generic issues such as improving visual representation. As developers seek more insightful code reviews [33, 40, 53, 86], it is crucial that the model is able to offer comments that reflect deeper issues. For this RQ, we manually evaluate the issue types discussed in the ELF models' code reviews.

4.2 Study Overview

The overview of our experimental design is presented in Figure 2. To answer all three RQs, we need to (1) construct a code review dataset tagged with ownership ratios to facilitate experience-aware fine-tuning, (2) fine-tune CodeReviewer's pre-trained checkpoint using each strategy, i.e., ELF, experience-aware oversampling, original and (3) evaluate all of the resulting models with respect to the metrics proposed for each of the RQs. To evaluate the degree of impact each strategy has on accuracy, RQ1 employs three accuracy metrics, i.e., BLEU-4, semantic equivalence and applicability. Whilst BLEU-4 is an automatic text matching metric that can be applied to the entire test set, semantic equivalence and applicability measure accuracy based on the actual meaning being conveyed in the review, which requires human level understanding. As a result, these two metrics are manually evaluated based on 100 random samples in the test set. To evaluate the degree of impact each strategy has on informativeness, RQ2 employs two metrics, i.e., feedback type, presence of explanation. Since these two metrics also rely on human level understanding to determine the level of information provided in the review, they are manually evaluated as well. Finally, to evaluate the degree of impact each strategy has on the issue types discussed, RQ3 requires manual evaluation to reliably categorise the content of the review. The details of the experimental design are discussed below (Sections 4.3–4.6).

4.3 Data Preparation

Dataset Selection. We use the CodeReviewer dataset provided by Li et al. [47], which is the benchmark dataset for the field of automated code reviews. Specifically, we use the code refinement set for review comment generation, as their original comment generation dataset did not retain pull request IDs, making them untraceable. The dataset represents a diverse set of software projects written in nine of the most popular programming languages on GitHub, including Python, Java,

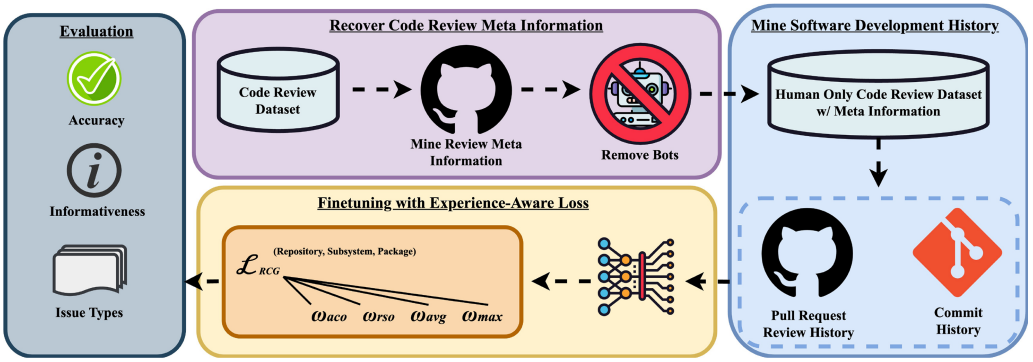

Fig. 2. Overview of our experimental design.

Go, C++, JavaScript, C, C#, PHP and Ruby. The training set was derived from 519 repositories, representing a subset of the top 10k most starred projects that contained more than 2.5k pull requests. The validation and test sets were derived from 307 repositories that contained between 1.5k and 2.5k pull requests. The size of the original dataset was 150,406 for training, 13,103 for validation and 13,104 for testing.

Meta Information Recovery. To retrieve meta information for each comment in the dataset, we used PyGithub[1] to retrieve the associated pull requests and identify the original comments using string matching. For each comment, we extracted the username and ID of the reviewer and the timestamp of when the comment was posted. Throughout this process, we identified 10,583 accounts who wrote reviews in the training set and 2,763 accounts in validation and test set. The reviews covered a timeframe between 2011 and 2022.

Preprocessing. Given that our focus is to build a review comment generation model that reflects the experienced perspective of human reviewers, we removed all bot accounts (e.g., CI bots, style checkers). This was achieved through two common methods [29]: (1) identification by the 'bot' suffix [94] and (2) identification by an established list of bots [30]. The identified bot accounts were manually inspected for false positives, which were subsequently retained. Finally, we discarded comments that provided code but without any natural language remarks. More specifically, we are referring to the GitHub code suggestion functionality that can be found in comments denoted with the "'*suggestion*'" format, where the reviewer's comment contains only a code edit of the original code hunk under review. This type of comment makes up approximately 5% of the refinement dataset [50], which caused severe mode collapse in the model when used for comment generation training. Without removing the code-only comments, the model's ability to provide real natural language comments can be degraded by only copy-and-pasting entire chunks of the submitted code, whilst inflating its performance on text matching-based results, e.g., BLEU-4. The final dataset included 141,259 examples for training, 12,406 for validation and 12,369 for testing. Table 1 presents statistics regarding the dataset filtering process.

Mining Software Development History. We collected meta information on all commits and pull requests for each of the 826 GitHub repositories. More specifically, for each commit, we retrieved the original author, a list of the changed files, and the timestamp using PyDriller [73] and for each pull request, we retrieved the GitHub users who left comments, a list of the changed files and the timestamp using PyGithub1. We observed that only some groups of developers used the GitHub review comment function, whilst others left reviews in the form of issue comments on the pull

[1] https://github.com/PyGithub/PyGithub.

Table 1. Dataset Overview

	Training	Validation	Test
Dataset Filtering (Reviews)			
Original Dataset Size	150,406	13,103	13,104
Untraceable	−618	−24	−41
Generated by Bot	−1,207	−41	−55
No Natural Language Comment	−7,322	−632	−639
Final Dataset Size	141,259	12,406	12,369
Mining Repository History			
# Repositories	519	300	300
# Reviewer Accounts	10,583	2,148	2,125
# Bot Accounts	9	3	4
# Past Commits	8,294,486	2,945,639	2,944,569
# Past Closed Pull Requests	3,478,749	606,148	605,649
Ownership Statistics (μ/σ)			
RSO (Repository)	0.21/0.21	0.31/0.24	0.31/0.24
ACO (Repository)	0.08/0.13	0.15/0.21	0.15/0.21
RSO (Subsystem)	0.25/0.23	0.35/0.26	0.35/0.26
ACO (Subsystem)	0.1 /0.14	0.17/0.22	0.17/0.22
RSO (Package)	0.31/0.27	0.39/0.29	0.38/0.29
ACO (Package)	0.12/0.19	0.18/0.24	0.18/0.24

request. Therefore, in addition to considering GitHub review comments such as in our previous work [50], we also consider issue comments as they demonstrate code reviewing activity [39]. In terms of commit history, we omit merge commits as they do not represent actual code authoring activity. Table 1 presents statistics regarding the repository history mining process.

Calculating Ownership Ratios. We calculate the ownership metrics with regard to each example (i.e., review comment) in the dataset. Although each reviewer may provide multiple reviews, the ownership metrics are calculated independently for each review comment. The rationale is that a reviewer's ownership coverage may change throughout time, hence our method reflects a version of that reviewer distilled at the timestamp of the review comment. The implementations are detailed in Algorithm 1 for ACO calculation and Algorithm 2 for RSO calculation, where RC denotes a review comment, D denotes a developer and G denotes the granularity level. Table 1 presents statistics regarding the calculated ownership ratios.

4.4 Model Fine-Tuning

Following CodeReviewer [47], we fine-tuned the models with a learning rate of $3e^{-4}$ using the AdamW optimizer. The training was set for 30 epochs at a batch size of 72. A beam search width of 10 was used during inference. Our hardware consisted of a single server with 32 CPUs, 256 GB RAM and four NVIDIA A100-80 GB GPUs. For fair comparison with the experience-aware models, we fine-tuned CodeReviewer on our filtered dataset. To incorporate our ELF and the previous experience-aware oversampling method, we altered the original Python scripts provided by Li et al. [47], which were implemented with PyTorch[2] and HuggingFace Transformers.[3] For a fair

[2]https://pytorch.org/.
[3]https://huggingface.co/.

Algorithm 1: Implementation of ACO

```
procedure ACO(D, G, RC)                                   ▷ ACO for one RC example
    α(D, G) ← 0
    C(G) ← 0
    for all commit ∈ G do
        τ1 ← RC timestamp
        τ2 ← commit timestamp
        if τ2 < τ1 then C(G) ← C(G) + 1                   ▷ # commits in G prior to RC
            if D ≡ commit author then α(D, G) ← α(D, G) + 1   ▷ # commits in G by D prior to RC
            end if
        end if
    end for
    return α(D, G)/C(G)          ▷ ratio of commits in G by D over total commits in G at τ1
end procedure
```

Algorithm 2: Implementation of RSO

```
procedure RSO(D, G, RC)                                   ▷ RSO for one RC example
    r(D, G) ← 0
    ρ(G) ← 0
    for all closed PR ∈ G do
        τ1 ← RC timestamp
        τ2 ← closed PR timestamp
        θ ← Set(reviewers of closed PR)
        if τ2 < τ1 then ρ(G) ← ρ(G) + 1                   ▷ # closed PRs in G prior to RC
            if D ∈ θ then r(D, G) ← r(D, G) + 1           ▷ # closed PRs in G reviewed by D prior to RC
            end if
        end if
    end for
    return r(D, G)/ρ(G)   ▷ ratio of closed PRs in G reviewed by D over total closed PRs in G at τ1
end procedure
```

comparison, we also re-implemented experience-aware oversampling using our new ownership values, which were calculated based on our newly mined repository histories (i.e., omitting merge commits, including issue comments). In total, we trained 16 models, including 12 ELF models, three experience-aware oversampling models, and one original CodeReviewer model.

4.5 Evaluation Metrics

Our evaluation consists of one automatic text matching metric and five manual evaluation tasks. Automatic text matching is measured on the entire test set, whilst the manual evaluation tasks are completed on a random sample of 100, achieving a 95% confidence level with 10% margin of error.

— *BLEU-4 (RQ1)*: To align with past research [47], we adopt the established Bilingual Evaluation Understudy metric [61] with up to 4-gram matching to benchmark the new approaches in terms of accuracy against the test set. We use the exact implementation provided by Li et al. [47], with stop words removed from the comments.

— *Semantic Equivalence (RQ1)*: This manual assessment of accuracy evaluates whether the model-generated comments possess the same intentions as the ground truth code reviews in the test set [45, 84, 85]. This metric disregards the degree of textual overlap, since the same intention can be expressed in various different ways. For example, in Figure 3, despite being worded differently, candidate A is considered semantically equivalent to the ground truth as the main intention of both comments is to change the error to a log warning.

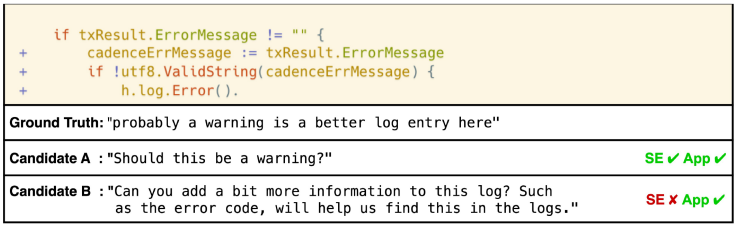

Fig. 3. Examples of semantically equivalent (SE) and applicable (App) comments.

— *Applicability (RQ1)*: This manual assessment of accuracy evaluates whether model-generated comments provide applicable reviews given the context of the submitted code change [47, 84, 85]. Since ground truth code reviews reflect only a subset of many possible issues with a particular piece of code, this evaluation captures the models' general ability to provide relevant code reviews. For example, in Figure 3, comment candidate B is not considered to be semantically equivalent to the ground truth; however, it is still applicable to the code change as the developer can make subsequent code improvements to address the comment. By definition, all comments that are semantically equivalent to the ground truth are also applicable.

— *Feedback Type (RQ2)*: To measure informativeness, this manual assessment categorises the code reviews into three distinct feedback types. We present them in order of most to least informative:
  – *Suggestion* [10, 33, 47]—Not only is an issue identified, but a solution is also proposed to address the issue.
  – *Concern* [33, 47]—An issue is identified or doubt is raised; however, no solution is provided.
  – *Confused Question* [13]—The comment demonstrates an inability to comprehend the code change.

  In Figure 4, the most uninformative feedback type is exemplified in the ground truth, which is a confused question that demonstrates an inability to understand the code change. Comment candidate A will be categorised as a concern because only a concern is raised, but no further solution is provided. Comment candidate B is considered as providing a suggestion because it recognises that the assert statements should not be removed and directly suggests to 'keep the "*d*" *tests*'.

— *Presence of Explanation (RQ2)*: This manual evaluation is a binary measure of whether the comment expresses their rationale. A comment that explains itself is considered to be more informative [40, 53, 68, 86]. For example, in Figure 4, we consider that comment candidate B has expressed their rationale as it describes why '"*d*" tests' should be kept '*to ensure that the code doesn't produce invalid JSON*'. On the contrary, both the ground truth and candidate A do not provide a rationale for their concerns or questions.

— *Comment Category (RQ3)*: This manual evaluation categorises the suggestions and concerns based on 15 established issue categories developed by past work [8, 10, 58, 86]. These categories are broadly segmented into three large classes, covering functional issues, evolvability issues and discussions. Functional issues are defects that can cause system failures at execution time, whilst evolvability issues are non-functional issues that affect the compliance, maintainability and understandability of the code. Discussions are dialogues that invoke thought regarding design directions and implementation choices. We only label comment category for comments previously annotated as applicable, as such, we do not consider the praise and false-positive categories like in prior work [86]. Comments that do not fit into the taxonomy are categorised

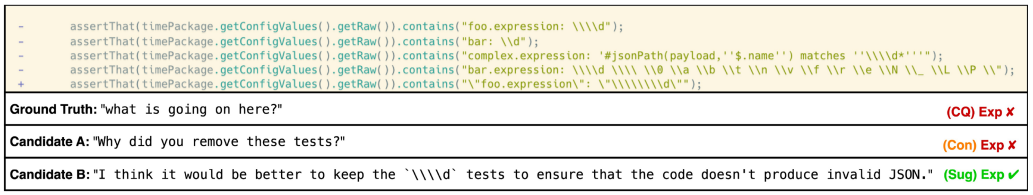

Fig. 4. Examples of suggestion (Sug) with explanation (Exp), concern (Con) and confused question (CQ).

Table 2. Code Review Comment Categories

Group	Category	Description
Functional	Functional Defect	A functionality is missing or implemented incorrectly, which often requires additional code or larger modifications
	Validation	Issues with detecting an invalid value and issues related to data sanitisation
	Logical	Issues with comparison operations, control flow, computations and other types of logical errors
	Interface	Issues when interacting with other parts of the software, e.g., existing code library, hardware device, database, OS
	Resource	Issues with the initialisation, manipulation and release of variables, memory, files and database
	Support	Issues related to support systems, libraries or their configurations
	Timing	Issues with incorrect thread synchronisation in shared resource settings
Evolvability	Solution Approach	Suggestions for alternate implementations, e.g., algorithms, data structures
	Documentation	Suggestions to improve code comments or documentation
	Organisation of Code	Suggestions for structural refactoring, e.g., collapse hierarchy, extract super class, inline function
	Alternate Output	Suggestions for improving error messages, toast messages, alerts and the returned values of a function
	Naming Convention	Suggestions for renaming software elements to comply with conventions
	Visual Representation	Suggestions for improving code readability, e.g., removing white spaces, blank lines, code rearrangements, indentation
Discussion	Question	Questions to understand design and implementation choices
	Design Discussion	Higher level discussions on design directions, design patterns and software architecture
Other		Comments that do not fit within the taxonomy

The code review comment categories and their respective descriptions are adopted from past work [8, 58, 86].

as other. The full list of code review comment categories and their respective descriptions are detailed in Table 2.

4.6 Manual Evaluation Process

To answer all three RQs, we rely on manually evaluated metrics, i.e., semantic equivalence (RQ1), applicability (RQ1), feedback type (RQ2), presence of explanation (RQ2) and comment categories (RQ3). We detail the evaluation process and discuss the inter-rater agreement for these five metrics below.

Overall Manual Evaluation Process. The manual evaluation was conducted by the first and sixth authors. Both annotators have previously been employed as software engineers and are currently pursuing a PhD in software engineering. One has 5 years of software development experience, whilst the other has over 10 years. The manual evaluation consisted of 8,000 annotations (100 generated comments × 16 models × five manual evaluation tasks). The annotators were provided

with a guideline including the definitions above, the submitted code hunk, the ground truth review comment, the generated reviews and the post-review code refinement. To mitigate incorrect annotations caused by a lack of familiarity with the software environments in the examples, the annotators may acquire more information from the original repositories and external resources such as Stack Overflow and official package/language documentation. For each manual evaluation task (1,600 annotations for 100 generated comments × 16 models), the two annotators independently completed two rounds of annotations where the first round consisted of 300 generated comments and the second round consisted of 200 generated comments. After each round, we measure inter-rater agreement and resolve all conflicts, resulting in a refined annotation guideline that both annotators agreed on [5]. After reaching an agreement rate that we were satisfied with [27] (≥0.8 Cohen's kappa [14]), the first author annotated the remaining 1,100 comments independently. Finally, the annotations were reviewed by the second and fifth authors. Note that we omit information regarding which models generated the comments during the annotation process to eliminate a confirmation bias, i.e., avoiding the evaluation results favouring particular models. Below, we summarise our annotation process for each evaluation task, including the inter-rater agreement achieved during the independent annotation rounds and the conflicting cases that were resolved.

Semantic Equivalence (RQ1). The annotators reached 89% agreement with a Cohen's kappa of 0.66 (substantial agreement) in the first round; the second round reached a satisfactory agreement rate of 94% with a Cohen's kappa of 0.82 (near perfect agreement). The most common type of conflict arose from the cases where the generated comments have semantically equivalent intention as the ground truth but suggest incorrect implementation, and where the generated comments have ambiguous intentions which are open to multiple interpretations. The comments with partial semantic equivalence were eventually labelled as not semantically equivalent as the suggested implementation would lead to the wrong fix. For the comments with ambiguous intentions, they were labelled as semantically equivalent if the subsequent ground truth code change was in the potential action space that could be elicited by the generated comment. Otherwise, they were labelled as not semantically equivalent.

Applicability (RQ1). The annotators reached 83% agreement with a Cohen's kappa of 0.67 (substantial agreement) in the first round; the second round reached a satisfactory agreement rate of 93% with a Cohen's kappa of 0.86 (near perfect agreement). The most common types of conflicts arose from cases where the generated comments required additional context and/or project knowledge to comprehend. For example, a generated comment suggested to rename a method '*fc_fit_scheduler*' to '*BasicTrainScheduler*', which can be considered as applicable. However, this suggestion is not suitable for a Python project as Pascal case violates PEP8 guidelines. These types of conflicts were resolved by consulting external resources in the context of the project.

Feedback Type (RQ2). The annotators reached 93% agreement with a Cohen's kappa of 0.86 (near perfect agreement) in the first round. Given that near perfect agreement was achieved, we omitted the second round and the first author performed the annotations on the remaining sampled comments. The most common types of conflicts arose from cases where the comments raised concerns in a question form which were similar to suggestions in question form. Code review suggestions are often provided in question form to be polite [20]; however, they can also be a request for confirmation. For example, '*Maybe close the "DeflaterOutputStream" here?*' and '*Do we need to close the "DeflaterOutputStream" here?*' are similar comments; however, the former is a polite suggestion that offers a solution, whilst the latter is a concern that requires other developers to clear their doubts. The conflicts were resolved by considering comments with '*Do we ...?*' questions as exhibiting uncertainty and therefore should be labelled as concern.

Presence of Explanation (RQ2). The annotators reached 99% agreement with a Cohen's kappa of 0.98 (near perfect agreement) in the first round. Given that near perfect agreement was achieved, the first

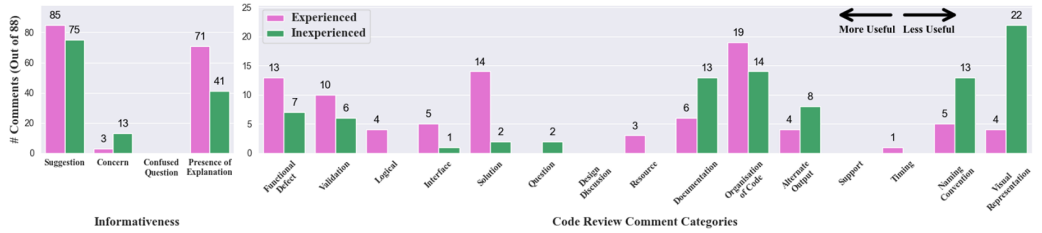

Fig. 5. Comparing informativeness and comment categories of code reviews by experienced versus inexperienced reviewers.

author performed the annotations on the remaining sampled comments without the second round. There was only one conflicting case where the comment asking '*Isn't there a complete_bipartite_graph function?*' can be considered as a self-embedded rationale when viewed as a suggestion, and as lacking a rationale when viewed as a concern asking about the code change itself. This case was resolved by considering with the context of the feedback type and code change.

Comment Category (RQ3). The annotators reached 88% agreement with a Cohen's kappa of 0.86 (near perfect agreement) in the first round. Given that near perfect agreement was achieved, we omitted this task from the second round. The most common types of conflicts arose from the misallocation of variable declaration changes to the resource category. This category includes variable initialisation changes, which distinctly focuses on problems related to resource allocation. Since changes such as improving the declared variable type do not alter the external behaviour of the code, we categorise them as organisation of code a.k.a refactoring type code reviews instead.

5 Preliminary Analysis on Code Review Comments and Reviewer Experience

Before evaluating the models trained with ELF, we first investigate whether the nature of code reviews varies with reviewer experience in reality. To investigate this, we compare the real distributions of informativeness and issue types discussed between the review comments of inexperienced and experienced reviewers. To this end, we sample code reviews of reviewers with polar opposite degrees of experience from the training set. We take extreme examples as there are no hard thresholds for determining general experience groups. Since experience likely exists on a spectrum, we abstain from using thresholds on intermediary ownership values to group code reviews. Thus, for this analysis, we consider inexperienced reviewers as those where all of their RSO and ACO values are two standard deviations below the mean or zero (RSO Repository = 0, ACO Repository = 0, RSO Subsystem = 0, ACO Subsystem = 0, RSO Package = 0, ACO Package = 0). Conversely, we consider experienced reviewers as those where all of their RSO and ACO values are two standard deviations above the mean (RSO Repository $\geq$ 0.63, ACO Repository $\geq$ 0.34, RSO Subsystem $\geq$ 0.71, ACO Subsystem $\geq$ 0.38, RSO Package $\geq$ 0.85, ACO Package $\geq$ 0.5). Based on these extreme thresholds, we extracted a pool of 1,031 reviews from inexperienced reviewers and 232 reviews from experienced reviewers. We randomly sample 88 examples from both pools of reviews, achieving a 95% confidence level with 10% margin of error. The first and sixth authors then manually annotated the sampled reviews together in terms of feedback type, presence of explanation and comment categories. The reviewer demographic information was omitted to prevent confirmation bias.

Figure 5 (left) shows the comparative results in terms of informativeness metrics. We find that experienced reviewers provided more suggestions and explanations than inexperienced reviewers. In particular, the difference in amount of explanations was substantial (Δ34%); this phenomenon is within expectation as code reviews are often used by senior developers as a platform for knowledge transfer within a project [4]. We find that explanations are prevalent when experienced reviewers

are discussing complex issues, e.g., deep functional defects, alternative implementation solutions. Figure 5 (right) shows the comparative results in terms of comment categories. These categories are ordered from most useful (left) to least useful (right) as rated by open-source developers [86]. We find that the code reviews from experienced developers tend to focus on more critical problems such as functional defects, validation issues, logical errors and incorrect usage of interfaces, whilst code reviews from inexperienced developers tend towards more trivial issues such as naming conventions and visual representation. Although inexperienced reviewers do discuss functional issues, the majority of issues were obvious syntactic bugs, e.g., '*Does this even work with a ";" inside those brackets?*', whilst experienced reviewers largely focused on deeper issues that require explanations, e.g., '*Can we make sure these two calls (ciphers and applicationProtocolConfig), or at least applicationProtocolConfig call, are made after the customizer is applied? Otherwise we would end up violating HTTP/2 specification IIUC*'. In general, the findings from our preliminary analysis coincides with past research, which further motivates the ELF method.

6 Results

In this section, we discuss the results with respect to the three RQs, which evaluate accuracy (Section 6.1), informativeness (Section 6.2) and issue types discussed (Section 6.3) in the comments generated by our ELF method.

For the count-based manual evaluation results on the 100 random samples, we are interested in measuring any significant improvements from our models compared to the original CodeReviewer model. To this end, we elect to report statistical significance based on the one-tailed two proportion Z-test [21]. The two proportion Z-test is used for comparing two proportions for significant differences, in this case one proportion would be the result of the original CodeReviewer model and the other proportion would be the result of one of our models. We select a one-tailed test as we are only interested in an improvement in one direction, e.g., significant increase in the number of suggestions generated out of 100 samples compared to the original CodeReviewer model.

6.1 (RQ1) What is the Impact of ELF on the Accuracy of Code Review Comment Generation Models?

Below, we present the accuracy results of our ELF models based on BLEU-4, semantic equivalence and applicability.

BLEU-4. All ELF models surpassed past methods in terms of matched n-grams. The best performing ELF models achieved 5% higher BLEU-4 scores than the original CodeReviewer model. As shown in Table 3, all ELF models (with different ownership values at different granularity levels) achieved higher BLEU-4 scores than the original CodeReviewer. In contrast, the experience-aware oversampling models achieved lower BLEU-4 scores. Comparing the performance of ELF models across different ownership values and granularities, all the models achieved comparable BLEU-4 scores ranging from 7.29 to 7.6. The highest performing ELF models were ω_{aco_Repo} and ω_{avg_Repo}, both recording without stop word BLEU-4 scores of 7.6, which is a +5% increase over the original CodeReviewer model. The results are also consistent when considering stop words in BLEU-4. These results suggest that ELF models can generate more comments that are textually similar to the ground truth than past models.

Semantic Equivalence. Our ELF models achieved results comparable to those of the original CodeReviewer model in terms of generating semantically equivalent comments to the ground truth. Table 3 shows that 20 comments out of 100 samples generated by the original CodeReviewer model are semantically equivalent to the ground truth. Our ELF models also achieve similar results, ranging from 17 to 23 comments that are semantically equivalent to the ground truth, where 7 of 12 ELF models generated more than 20 of such comments. In contrast, all experience-aware oversampling models generated less than 20 semantically equivalent comments. The highest performing ELF

Table 3. BLEU-4 on the Entire Test Set, Semantic Equivalence and Applicability on 100 Random Samples

			Exp-Aware Oversampling			ELF											
						ω_{aco}			ω_{rso}			ω_{avg}			ω_{max}		
		ORG	*MRMA*	*MR*	*MA*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*
Test Set	$B4_{w/o\ Stopwords}$	7.27	6.87 ↓	6.71 ↓	7.11 ↓	**7.6** ↑	7.56 ↑	7.46 ↑	7.45 ↑	7.57 ↑	7.55 ↑	**7.6** ↑	7.36 ↑	7.45 ↑	7.43 ↑	7.29 ↑	7.38 ↑
	$B4_{w/\ Stopwords}$	5.81	5.59 ↓	5.5 ↓	5.72 ↓	6.09 ↑	6.07 ↑	6.02 ↑	5.92 ↑	**6.1** ↑	6.02 ↑	6.07 ↑	5.95 ↑	5.96 ↑	5.93 ↑	5.82 ↑	5.99 ↑
Samples	SE	20	14 ↓	18 ↓	19 ↓	17 ↓	21 ↑	19 ↓	**23** ↑	**23** ↑	20	22 ↑	20	22 ↑	22 ↑	22 ↑	19 ↓
	App	42	40 ↓	38 ↓	44 ↑	37 ↓	**54*** ↑	53 ↑	52 ↑	53 ↑	53 ↑	44 ↑	43 ↑	46 ↑	48 ↑	46 ↑	44 ↑

ORG, Original Code Reviewer; *MRMA*, Major Reviewer Major Author; *MR*, Major Reviewer; *MA*, Major Author; *Repo*, Repository; *Sys*, Subsystem; *Pkg*, Package; B4, BLEU-4; SE, Semantic Equivalence; App, Applicablility.

Bold indicates the best score, Increased from *ORG* (↑), Decreased from *ORG* (↓), $p < 0.05$ (*).

models were ω_{rso_Repo} and ω_{rso_Pkg}, both achieving 23 correct matches. These results suggest that the use of experience-aware methods does not have a large impact on the semantic equivalence of model-generated comments towards the ground truth. However, it is important to note that this evaluation did not consider the quality of generated comments. Given that our key goal is to improve the quality of generated comments by aligning with the comments of experienced reviewers, we did not expect to achieve an improvement in terms of semantic equivalence towards the general population of the dataset. To this end, we assess the quality of generated comments using the subsequent metrics.

Applicability. Our ELF models can generate more comments that are applicable to the code changes than the original CodeReviewer model. The top performing ELF model generated 29% more applicable comments than the original CodeReviewer model. Table 3 shows that 42 comments out of 100 samples generated by the original CodeReviewer model were applicable to the code change. All ELF models apart from ω_{aco_Repo} generated more applicable comments than the original CodeReviewer model, ranging from 43 to 54 applicable comments. For experience-aware oversampling, only *MA* could outperform the original CodeReviewer model. In total, 8 of 12 ELF models generated more applicable comments than all past techniques. Comparing across ELF strategies, we found that ω_{rso} models were consistently high performing, generating between 52 and 53 applicable comments. Overall, ω_{aco_Sys} was the top performer with 54 applicable comments, which is a statistically significant increase of +29% over the original CodeReviewer model.

RQ1: All ELF models surpassed past methods in terms of BLEU-4, achieving up to +5% increase over the original CodeReviewer model. Overall, all ELF models achieved comparable results to the original CodeReviewer model in terms of semantically equivalent comments. Our ELF models were able to generate more applicable comments than all past methods, achieving up to +29% increase over the original CodeReviewer model.

6.2 (RQ2) What is the Impact of ELF on the Informativeness of Code Review Comment Generation Models?

Feedback Type. Our ELF models provided more suggestions than all past methods. The top performing ELF model generated 56% more suggestions than the original CodeReviewer model. Table 4 shows that 27 comments out of 100 samples generated by the original CodeReviewer model were suggestions (64% of its applicable comments). With the exception of ω_{aco_Repo} and ω_{avg_Sys}, all ELF models provided more suggestions than the original CodeReviewer model, ranging from 28 to 42 suggestions. The experience-aware oversampling models also outperformed the original CodeReviewer model in terms of number of generated suggestions; however, the improvements were not significant. Overall, 5 of 12 ELF models surpassed all past techniques in terms of generated suggestions, all of which were ω_{aco} and ω_{rso} models. Comparing across strategies, we found that ω_{rso} models showed the most consistent improvements, generating between 34 and 37 suggestions. Comparing

Table 4. Feedback Type and Presence of Explanation on 100 Random Samples

			Exp-Aware Oversampling			ELF											
						ω_{aco}			ω_{rso}			ω_{avg}			ω_{max}		
	GT	*ORG*	*MRMA*	*MR*	*MA*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*
Sug	87	27	31 ↑	28 ↑	30 ↑	25 ↓	34 ↑	**42***↑	34 ↑	35 ↑	37 ↑	29 ↑	24 ↓	31 ↑	30 ↑	28 ↑	30 ↑
Con	10	8	8	9 ↑	11 ↑	10 ↑	**17***↑	9 ↑	14 ↑	15 ↑	11 ↑	11 ↑	16*↑	12 ↑	12 ↑	14 ↑	12 ↑
CQ	3	7	**1***↓	**1***↓	3 ↓	2*↓	3 ↓	2*↓	4 ↓	3 ↓	5 ↓	4 ↓	3 ↓	3 ↓	6 ↓	4 ↓	2*↓
Exp	68	8	16*↑	16*↑	**20***↑	10 ↑	11 ↑	15 ↑	11 ↑	18*↑	11 ↑	11 ↑	12 ↑	13 ↑	12 ↑	11 ↑	15 ↑

GT, Ground Truth; *ORG*, Original Code Reviewer; *MRMA*, Major Reviewer Major Author; *MR*, Major Reviewer; *MA*, Major Author; *Repo*, Repository; *Sys*, Subsystem; *Pkg*, Package; Sug, Suggestion; Con, Concern; CQ, Confused Question; Exp, Explanation.

Bold indicates the best score, Increased from *ORG* (↑), Decreased from *ORG* (↓), $p < 0.05$ (*).

across granularities, we found that package level models tended to provide the most suggestions, as they were consistently the highest performer across all four strategies for this task. The top performer, ω_{aco_Pkg}, generated 42 suggestions (79% of its applicable comments), yielding a statistically significant increase of +56% over the original CodeReviewer model.

All ELF models generated more concerns and fewer confused questions than the original CodeReviewer model. The top performing ELF models generated 71% less confused questions than the original CodeReviewer model. Table 4 shows that eight comments out of 100 samples generated by the original CodeReviewer model were concerns. All ELF models generated more concerns than the original CodeReviewer model, ranging from 9 to 17 concerns generated. Similarly, both *MR* and *MA* also provided more concerns than the original CodeReviewer model, however, the differences were not significant. Interestingly, subsystem level views tended to produce the most concerns across all four strategies for this task. Table 4 shows that seven comments out of 100 samples generated by the original CodeReviewer model were confused questions (17% of its applicable comments). All ELF models manifested less confusion than the original CodeReviewer model, ranging from two to six confused questions generated. The top performing ELF models, ω_{aco_Repo}, ω_{aco_Pkg} and ω_{max_Pkg} generated only two confused questions each (5% of their applicable comments), which were statistically significant decreases of −71% against the original CodeReviewer model. Similarly, all experience-aware oversampling models also exhibited less confusion, with *MRMA* and *MR* generating only one confused question each.

Presence of Explanation. We found that all ELF models tended to provide rationales more frequently than the original CodeReviewer model. The top performing ELF model generated 125% more comments with rationales than the original CodeReviewer model. Table 4 shows that eight comments out of 100 samples generated by the original CodeReviewer model contained a rationale (19% of its applicable comments). All ELF models surpassed the original CodeReviewer model in this regard, generating between 10 and 18 comments with rationales. Amongst the ELF models, ω_{rso_Sys} was the top performer, generating 18 of such comments (34% of its applicable comments), demonstrating a statistically significant increase of +125% over the original CodeReviewer model. Overall, experience-aware oversampling models still provided the most explanations, with *MA* generating up to 20 comments with rationales.

RQ2: Our ELF models were able to provide more suggestions than all past methods, achieving up to +56% increase over the original CodeReviewer model. Similar to experience-aware oversampling, all ELF models also generated more concerns with fewer confused questions, achieving up to −71% decrease compared to the original CodeReviewer model in terms of confused questions generated. All ELF models provided explanations more frequently, achieving up to +125% increase over the original CodeReviewer model.

Table 5. Code Review Comment Categories on 100 Random Samples

			Exp-Aware Oversampling			ELF											
						ω_{aco}			ω_{rso}			ω_{avg}			ω_{max}		
	GT	*ORG*	*MRMA*	*MR*	*MA*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*	*Repo*	*Sys*	*Pkg*
Total Functional Issues	35	7	9 ↑	9 ↑	7	10 ↑	12 ↑	13 ↑	10 ↑	11 ↑	12 ↑	8 ↑	9 ↑	**16*** ↑	11 ↑	9 ↑	11 ↑
Functional Defect	6	2	2	1 ↓	3 ↑	1 ↓	3 ↑	**4** ↑	3 ↑	2	2	2	1 ↓	2	2	1 ↓	2
Validation	6	1	2 ↑	**4** ↑	0 ↓	1	1	2 ↑	1	2 ↑	1	0 ↓	2 ↑	1	2 ↑	1	3 ↑
Logical	5	0	2 ↑	1 ↑	1 ↑	3* ↑	**4*** ↑	2 ↑	1 ↑	2 ↑	2 ↑	2 ↑	2 ↑	3* ↑	2 ↑	2 ↑	0 ↓
Interface	7	1	0 ↓	1	0 ↓	0 ↓	0 ↓	**2** ↑	1	0 ↓	1	0 ↓	1	**2** ↑	1	0 ↓	**2** ↑
Resource	6	2	2	0 ↓	2	3 ↑	2	1 ↓	3 ↑	3 ↑	5 ↑	3 ↑	2	**6** ↑	3 ↑	4 ↑	2
Support	1	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
Timing	4	0	0	**1** ↑	0	**1** ↑	**1** ↑	**1** ↑	0	**1** ↑	0	0	0	**1** ↑	0	0	**1** ↑
Total Evolvability Issues	57	24	26 ↑	23 ↓	28 ↑	19 ↓	27 ↑	**29** ↑	27 ↑	27 ↑	27 ↑	24	21 ↓	19 ↓	23 ↓	27 ↑	23 ↓
Solution Approach	7	4	2 ↓	3 ↓	4	1 ↓	3 ↓	2 ↓	3 ↓	4	3 ↓	3 ↓	3 ↓	2 ↓	3 ↓	3 ↓	**5** ↑
Documentation	7	0	3* ↑	2 ↑	2 ↑	3* ↑	4* ↑	4* ↑	4* ↑	**6*** ↑	**6*** ↑	3* ↑	2 ↑	1 ↑	2 ↑	3* ↑	2 ↑
Organisation of Code	20	3	6 ↑	3	**9*** ↑	2 ↓	2 ↓	5 ↑	4 ↑	3	2 ↓	5 ↑	5 ↑	2 ↓	4 ↑	7 ↑	2 ↓
Alternate Output	6	4	3 ↓	4	2 ↓	1 ↓	**7** ↑	6 ↑	3 ↓	2 ↓	4	1 ↓	2 ↓	4	4	1 ↓	2 ↓
Naming Convention	12	7	**9** ↑	7 ↑	4 ↓	7	6 ↓	8 ↑	**9** ↑	**9** ↑	**9** ↑	7	5 ↓	7	7	**9** ↑	8 ↑
Visual Representation	5	6	3 ↓	4 ↓	**7** ↑	5 ↓	5 ↓	4 ↓	4 ↓	3 ↓	3 ↓	5 ↓	4 ↓	3 ↓	3 ↓	4 ↓	4 ↓
Total Discussion	3	4	4	5 ↑	5 ↑	6 ↑	**11*** ↑	8 ↑	10* ↑	**11*** ↑	8 ↑	7 ↑	10* ↑	7 ↑	7 ↑	6 ↑	8 ↑
Question	1	4	4	2 ↓	4	6 ↑	**10*** ↑	7 ↑	9 ↑	**10*** ↑	7 ↑	6 ↑	**10*** ↑	7 ↑	7 ↑	6 ↑	6 ↑
Design Discussion	2	0	0	**3*** ↑	1 ↑	0	1 ↑	1 ↑	1 ↑	1 ↑	1 ↑	1 ↑	0	0	0	0	2 ↑
Other	2	0	0	0	**1** ↑	0	**1** ↑	**1** ↑	**1** ↑	**1** ↑	**1** ↑	**1** ↑	0	**1** ↑	**1** ↑	0	0

GT, Ground Truth; *ORG*, Original Code Reviewer; *MRMA*, Major Reviewer Major Author; *MR*, Major Reviewer; *MA*, Major Author; *Repo*, Repository; *Sys*, Subsystem; *Pkg*, Package.

, Functional Issue; , Evolvability Issue; , Discussion.

Bold indicates the best score, Increased from *ORG* (↑), Decreased from *ORG* (↓), $p < 0.05$ (*).

6.3 (RQ3) What Issue Types Are Discussed in the ELF Models' Code Reviews?

Functional Issues. Our ELF models generated more comments related to functional issues than all past methods. The top performing model identified +129% more functional issues than the original CodeReviewer model. Table 5 shows that seven comments out of 100 samples generated by the original CodeReviewer model were related to functional issues (17% of its applicable comments). In this regard, 9 of 12 ELF models surpassed all past methods, generating between 10 and 16 functional issue-related comments. In terms of experience-aware oversampling methods, both *MRMA* and *MR* outperformed the original CodeReviewer model; however, the improvements were not significant. Comparing across the granularities, we found that package level models tended to provide more comments related to functional issues; however, there was no observable pattern in the specific category types that yielded the improvement. Compared to the original CodeReviewer model, we found that ω_{aco_Pkg} was able to elicit five new comments related to high priority issues, such as logical, validation and functional defects. The top performing model for this task was ω_{avg_Pkg}, which generated 16 comments related to functional issues (35% of its applicable comments), demonstrating a statistically significant increase of +129% over the original CodeReviewer model. Most of this improvement can be attributed to its ability to identify new logical errors and resource issues, i.e., incorrect initialisation, manipulation and release.

Evolvability Issues. Our ELF models generated more comments related to evolvability issues than the original CodeReviewer model. Specifically, all ELF models unanimously identified more documentation-related issues than the original CodeReviewer model. Table 5 shows that 24 comments out of 100 samples generated by the original CodeReviewer model were related to evolvability issues. In comparison, 6 of 12 ELF models generated more evolvability-related comments, ranging from 27 to 29 of such comments. For experience-aware oversampling, both *MRMA* and *MA* also slightly outperformed the original CodeReviewer model. Comparing across strategies, we found that ω_{rso} models yielded the most consistent improvements, achieving a +13% increase over the original code reviewer model. These improvements can be attributed to their ability to identify opportunities for improving documentation and code element renaming. Interestingly, every ELF model identified

more documentation related issues, where ω_{aco} and ω_{rso} models all achieved statistically significant improvements, reaching up to +600% increase over the original CodeReviewer model. All ELF models produced fewer comments related to trivial issues, i.e., visual representation, achieving up to −50% decrease from the original CodeReviewer model. Overall, ω_{aco_Pkg} yielded the most improvement, with a +21% increase over CodeReviewer, identifying more improvement opportunities for a wide array of evolvability issues (four of six categories).

Discussion. All ELF models outperformed past methods in terms of discussion invoking comments. The top performing ELF models generated 150% more questions concerning implementation choices than the original CodeReviewer model. Table 5 shows that three comments out of 100 samples generated by the original CodeReviewer model were discussions. In contrast, all ELF models outperformed past methods, generating between 6 and 11 discussions. In this aspect, experience-aware oversampling demonstrated negligible overall difference compared to the original CodeReviewer model. The majority of improvements from the ELF models can be attributed to an increase in questions concerning implementation choices. In particular, ω_{aco_Sys}, ω_{rso_Sys} and ω_{avg_Sys} exhibited statistically significant increases of +150% over CodeReviewer in terms of these types of questions.

RQ3: Our ELF models identified more functional issues than all past methods, achieving up to +129% increase over the original CodeReviewer model. Our ELF models found more evolvability issues. Specifically, all ELF models identified more opportunities for improving documentation, achieving up to +600% increase over the original CodeReviewer model. All ELF models generated more questions concerning implementation choices than past methods, achieving up to +150% increase over the original CodeReviewer model.

7 Analysis and Discussion

In this section, we provide further analysis of our results. First, we investigate the distribution of ownership values and how they are related to each other (Section 7.1). Second, we compare the quality of code reviews generated by ELF against real code reviews (Section 7.2). Third, we discuss the BLEU-4 score improvements of our ELF models (Section 7.3). Next, we discuss the value of considering reviewer experience at different granularities (Section 7.4). Finally, we compare the performances of different ELF strategies (Section 7.5) and conduct a sensitivity analysis (Section 7.6).

7.1 How Are the Different Ownership Values Distributed and What Is Their Relationship to Each Other?

A difference in behaviour amongst the ELF models is only possible if reviewers' ownership ratios vary between the authoring and reviewing perspective and across the different granularities. Thus, we first investigate the distributions and relationships of the different ownership ratios to gain initial insight into why different ELF models may generate different code review comments.

The kernel density estimates of the ownership ratios in Figure 6 demonstrate that reviewers tend to have higher RSO than ACO. In fact, many reviews are provided by experienced developers who contribute mainly via reviewing code, rather than by writing code, which aligns with past findings [76]. Interestingly, a rise in ACO values consistently comes with a rise in RSO values, indicating that developers who are responsible for a large portion of commits tend to also be responsible for a larger portion of code reviews. Comparing across the dataset splits, it is evident that ownership ratios in the training set are more concentrated around smaller values as opposed to the validation and test sets. This aligns with intuition since it is more difficult for an individual to gain high ownership coverage in large projects that have more contributors. We find that ACO and RSO values

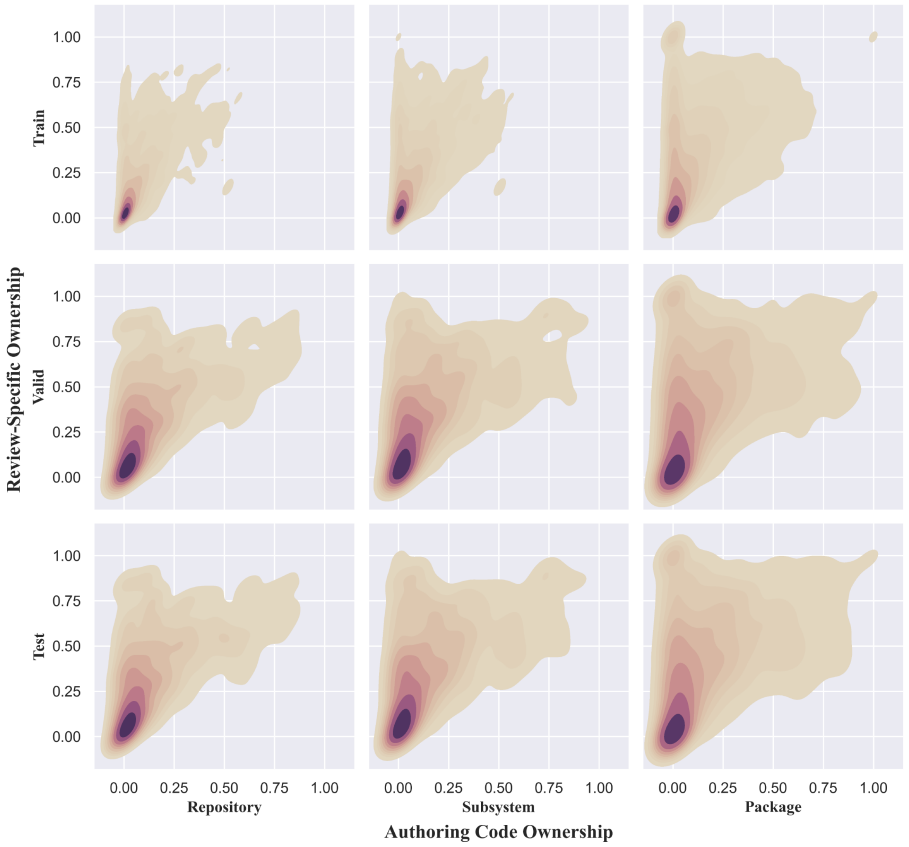

Fig. 6. Kernel density estimates of ACO and RSO at repository, subsystem and package level.

consistently increase as we consider a more refined granularity, indicating that many developers have more specialised coverage within the project. We report the Pearson correlation for ACO in the training set between repository and subsystem ($\rho = 0.85$), between subsystem and package ($\rho = 0.69$) and finally between repository and package ($\rho = 0.58$). We also report the Pearson correlation for RSO in the training set between repository and subsystem ($\rho = 0.67$), between subsystem and package ($\rho = 0.74$) and finally between repository and package ($\rho = 0.67$). 💡 The divergence in both correlation and magnitude of ownership ratios between the different granular views indicate potential for varying signals. As such, a difference in model behaviour when employing ELF from different granularities was within our expectation.

7.2 How Do Code Reviews Generated by ELF Compare with the Ground Truth in Terms of Quality?

Whilst our main results section compares reviews generated by the different review comment generation models, this section compares the reviews generated by the best performing models to the ground truth reviews provided by human reviewers. The 100 ground truth examples were randomly sampled from the distribution of the test set; thus, it includes reviewers from a variety of experience levels. This is particularly relevant as code reviewing in reality is not restricted to experienced reviewers only. The main reason for this design was to compare the nature of

Fig. 7. Example of an ELF model generating a natural language code review with embedded code snippet.

code reviews generated from the different techniques against the nature of real code reviews that developers are receiving. We report the summary statistics for the ownership values of these 100 samples (RSO Repository $(\mu, \sigma) = (0.26, 0.24)$, ACO Repository $(\mu, \sigma) = (0.14, 0.23)$, RSO Subsystem $(\mu, \sigma) = (0.31, 0.26)$, ACO Subsystem $(\mu, \sigma) = (0.16, 0.24)$, RSO Package $(\mu, \sigma) = (0.35, 0.29)$ and ACO Package $(\mu, \sigma) = (0.19, 0.26)$). In terms of feedback type, the proportions of suggestions in the 100 samples is 87%. For the best performing ELF strategy in this regard, ω_{aco_Pkg}, only 79% of their applicable comments were suggestions. The proportion of code reviews that had explanations in the 100 samples is 68%. For the best performing ELF strategy in this regard, ω_{rso_Sys}, only 34% of their applicable comments contained explanations. In terms of comment categories, 35% of the 100 samples discuss critical functional issues. For the best performing ELF strategy in this regard, ω_{avg_Pkg}, 35% of their applicable comments also discuss critical functional issues. Although the ELF method yields similar behaviour with the real code reviews in terms of proportion of functional issues discussed, they still fall short in terms of informativeness. Thus, reaching this informativeness standard still remains an objective for current models. Future work should also progressively track model progress against more experienced reviewers.

7.3 Why Do ELF Models Generate More Textually Similar Reviews to the Ground Truth?

To investigate the unanimous improvement of ELF models in terms of BLEU-4, we manually analysed the top 10 examples for each ELF model in terms of the largest BLEU-4 delta against the original CodeReviewer model.

We observe that all ELF models tended to provide code snippets with exact implementation suggestions embedded within their natural language comment as opposed to the original model which tended to provide natural language comments that occasionally included code elements. Thus, when ELF models were semantically equivalent to the ground truth, they often provided close to exact match code implementations to explain their suggestions, which achieves near perfect BLEU-4 results, as demonstrated in Figure 7. In the cases where ELF models did not achieve semantic equivalence, they were able to locate the exact code that was considered problematic by the ground truth comment, which also resulted in high n-gram matches. This finding is significant given that using code snippets to explain implementations is rarely done in code reviews and a method used by experienced reviewers to guide new developers in the project [99]. Often used for suggestions and citations, reviewers employ these code snippets when the intended revision cannot be clearly described in words. This ensures that ideas can be quickly evaluated with less room for miscommunication. Given that these types of comments are well received by developers, we consider this an improvement in the quality of generated comments. We emphasise that the ELF models have learned this behaviour without losing the ability to generate natural language reviews; this can be contrasted with the previously discussed issue where models learned only to copy and paste code inputs due to problematic examples in the dataset that included only code suggestions without any natural language component.

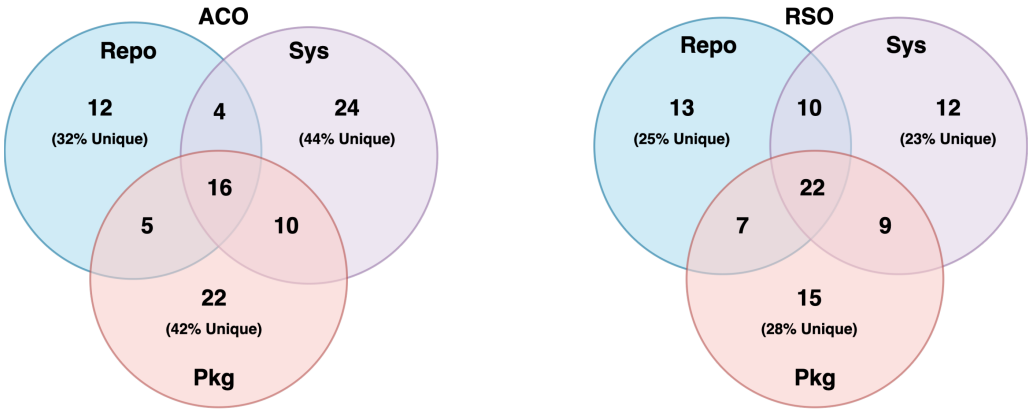

Fig. 8. Diversity of applicable comments generated by ACO- and RSO-based strategies in terms of comment category.

7.4 What Is the Value of Considering Reviewer Experiences from Different Granularities?

As observed in the results, all granularities perform similarly in terms of BLEU-4, semantic equivalence and applicability. However, similar metric results do not indicate that the different views are generating the same comments. To further explore the value of employing different granularities, we examine whether there is a difference in generated comments within each of the four proposed strategies.

First, we investigate the degree to which different granularities are achieving semantic equivalence on the same ground truth examples. For ω_{aco}, we find that the three granular views can cover 29 semantically equivalent comments together with $\frac{10}{29}$ being mutually inclusive and $\frac{11}{29}$ attributable to only one unique view. For ω_{rso}, we find that the three granular views can cover 34 semantically equivalent comments together with $\frac{10}{34}$ being mutually inclusive and $\frac{12}{34}$ attributable to only one unique view. For ω_{avg}, we find that the three granular views can cover 32 semantically equivalent comments together with $\frac{10}{32}$ being mutually inclusive and $\frac{10}{32}$ attributable to only one unique view. For ω_{max}, we find that the three granular views can cover 32 semantically equivalent comments together with $\frac{12}{32}$ being mutually inclusive and $\frac{13}{32}$ attributable to only one unique view. 💡 For all strategies, more than half of the total semantically equivalent matches cannot be simultaneously covered by all three granular views, a substantial portion of which can only be covered by one particular view. To further explore the diversity of the generated comments, we conducted the same analysis on the types of elicited comments, which is displayed in Figure 8. Two applicable code reviews of the same code change submission are considered to be different if they fall under different code review categories. For ω_{aco}, we find that 32%, 44% and 42% of the applicable comments generated by repository, subsystem and package level views, respectively, were completely unique. For ω_{rso}, the proportion of applicable comments that were completely unique are 25%, 23% and 28%, respectively. 💡 The high percentage of uniquely generated comments indicates that the varying ownership ratios between the granularities in fact do elicit diverging perspectives. As such, we find that there is value in considering all three granularities when training automated code review models with ELF.

7.5 Which ELF Strategies Elicit the Most Improvement in Terms of Code Review Comment Quality?

Comparing across the strategies, we find that all three models of ω_{rso} are consistently high performing across the majority of the tasks, i.e., semantic equivalence, applicability, suggestions

provided, identified evolvability issues and discussion invoking comments. However, the subsystem and package level models of ω_{aco} are consistently high performing across nearly all tasks apart from semantic equivalence. 💡 In contrast, we find that both ω_{avg} and ω_{max} strategies rarely bring additional value, indicating that experience types should be considered separately during training.

Since the subsystem and package level models of both ω_{aco} and ω_{rso} are top performing, we analyse their overlap in issue types discussed in the generated comments. We find that the overlap between ω_{aco_Pkg} and ω_{rso_Pkg} is only 22% compared to the overlap between ω_{aco_Sys} and ω_{rso_Sys} at 37%. 💡 This indicates that the package level models of ω_{aco} and ω_{rso} are not only more accurate and informative, but offer the most diverse range of code reviews. Compared to CodeReviewer, ω_{aco_Pkg} is able to identify more high priority issues related to functional defects, validation errors and logical faults, which are the top three most useful code review categories as rated by open-source developers [86]. We highlight the significance of this finding as code reviews rarely find functionality defects in reality [15]. This demonstrates that targeting reviewers' coding experience at the package level can pinpoint critical code reviews during model training. To complement ω_{aco_Pkg}, ω_{rso_Pkg} is able to catch more resource-related issues. ω_{aco_Pkg} improves on a wide variety of evolvability issues, i.e., documentation, organisation of code (refactoring), alternate output and naming convention. Unlike visual representation issues (formatting), these evolvability issues are often beyond the scope of traditional static analysis tools [89], making them highly valuable types of code reviews to generate. More specifically, improving the organisation of code can help ameliorate low evolvability in systems, which hinders developer productivity when adding features or fixing bugs [6, 71]. On the other hand, ω_{rso_Pkg} excels at improving documentation, which can aid the comprehensibility of the programs [60]. Overall, we find that both ω_{aco} and ω_{rso} strategies are the most effective, especially at the package level. Given the diverse nature of these two models, it is highly synergetic to integrate both models together.

7.6 How Sensitive Is the ELF Method to Weighting Strategy Selection?

Although the ELF method removes the need to tune ownership thresholds, there still exists the need to select a particular loss function weighting. Through our results, we identified ω_{aco_Pkg} and ω_{rso_Pkg} to be top performers; however, it remains unclear how sensitive the ELF method is to strategy selection overall. To this end, we analyse the performance gaps between the different weighting strategies, using the same statistical testing as the main results. Figure 9 shows the distribution of results across all ELF weighting strategies. When not considering the aforementioned top performers, we find that results can be sensitive to weighting strategy selection in terms of informativeness and issue types discussed, whilst accuracy is generally consistent. We discuss findings from the sensitivity analysis in more detail below.

To reflect the impact of weighting strategy selection on accuracy, we report the gap between the best and worst performing ELF models in terms of BLEU-4, semantic equivalence and applicability. The distribution of accuracy results is presented through the pink boxplots in Figure 9. In terms of BLEU-4, we find that the differences in performance are minimal, where the maximum gap in BLEU-4 with and without stop words are merely $\Delta 0.28$ $(6.1 - 5.82)$ and $\Delta 0.31$ $(7.6 - 7.29)$, respectively. Similarly, we find statistically insignificant differences in semantic equivalence, where the maximum gap is $\Delta 6$ $(23 - 17)$ in correct matches. In contrast, the ω_{aco_Repo} variant yields a statistically significant decrease of $\Delta 17$ $(54 - 37)$ in applicable comments from the top performer, whilst the rest of the variants yield insignificant differences. With the exception of the case mentioned above, we find that accuracy of the ELF method vary minimally to weighting strategy selection.

To reflect the impact of weighting strategy selection on informativeness, we report the gap between the best and worst performing ELF models in terms of feedback type and presence of explanation. The distribution of informativeness results are presented through the brown boxplots

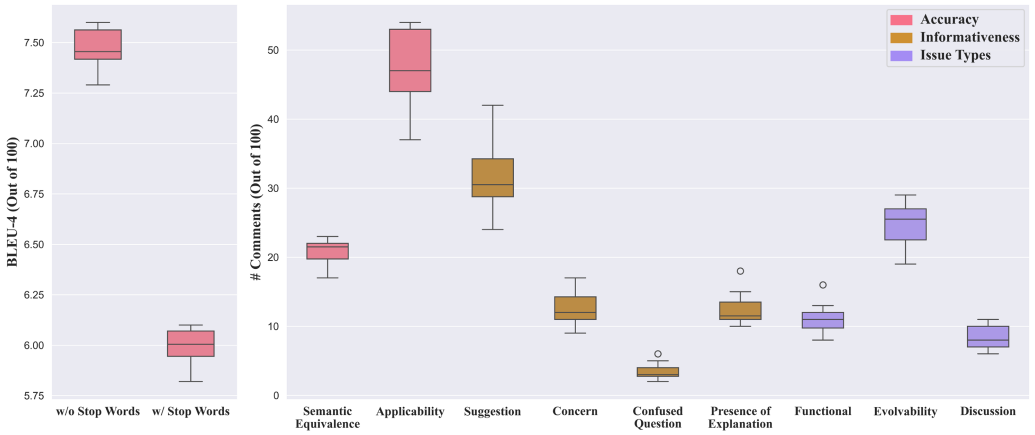

Fig. 9. Distribution of results across all ELF strategies.

in Figure 9. In terms of feedback type, we find that the variation in both suggestions and concerns provided can be statistically significant, with a maximum gap of $\Delta 18$ $(42 - 24)$ in suggestions and $\Delta 8$ $(17 - 9)$ in concerns. In contrast, we find that amount of confused questions generated only varied by a maximum of $\Delta 4$ $(6 - 2)$, which is insignificant. Similarly, we find that the variation in amount of generated explanations is also statistically insignificant, with a maximum gap of $\Delta 8$ $(18 - 10)$. Thus, the ELF method varies minimally to weighting strategy selection for certain informativeness measures, such as number of confused questions and explanations generated; however, this selection causes large variations in the number of suggestions and concerns generated, when not considering ω_{aco_Pkg} and ω_{rso_Pkg}.

To reflect the impact of weighting strategy selection on issue types discussed, we report the gap between the best and worst performing ELF models in terms of the categories of generated comments. These distributions are presented through the purple boxplots in Figure 9. In terms of functional issues, we find that ω_{avg_Repo} can yield a statistically significant decrease of $\Delta 8$ $(16 - 8)$ from the top performer, whilst the rest of the variants yield insignificant differences. For evolvability issues, we find that variations can be significant, where the maximum gap is $\Delta 10$ $(29 - 19)$. In contrast, the amount of generated discussions vary minimally, where the maximum gap is $\Delta 5$ $(11 - 6)$. In general, the ELF method varies minimally to weighting strategy selection in terms of functional issues and discussions. On the other hand, this selection can cause significant variations in number of comments generated regarding evolvability issues, when not considering ω_{aco_Pkg} and ω_{rso_Pkg}.

Since behaviour in terms of various informativeness measures and issues types discussed can vary due to the selected weighting strategy, future researchers and practitioners should preliminary evaluate the different strategies on any validation set of interest before selection. Nevertheless, ω_{aco_Pkg} and ω_{rso_Pkg} remain consistently top performing across all evaluation metrics in our experiments.

8 Broader Implications

We now discuss the broader implications of our study.

Practitioners. In theory, if code review comment generation models could successfully mimic experienced reviewers in disseminating expert knowledge during the review, the usefulness of the tool would better align with preferences of developers [40]. However, given the non-trivial

nature of the task, the state of open-source code review comment generation models is still far from practical, as exhibited by the large portion of non-applicable comments being generated. Whilst our released model checkpoints can be directly deployed for inference with frameworks that support the T5 language model architecture, software developers and project maintainers should proceed with caution, given the tool is in early development and has not demonstrated completely reliable levels of accuracy. Nevertheless, the ELF method presents a data efficient way to improve code review comment generation models, which is especially critical given the sparsity of such corpora. For model developers who want to leverage ELF to train their code review comment generation models, there are two base requirements that need to be fulfilled, (1) abundant code review data from mature software projects and (2) accurate version control history. An abundance of code review data is needed such that generalisable patterns can be learned, and mature software projects are required such that experienced reviewers can exist. For a single project, the amount of code review data from experienced reviewers is often insufficient; thus, practitioners should incorporate mature projects that have similar characteristics to their target project, e.g., application domain, software engineering standards, programming languages. Whilst there is no established threshold for determining when a project can be considered mature, the rule of thumb used to collect the training set was more than 2.5k pull requests [47]. In terms of accurate version control history, practitioners can benefit from projects where a complete record of the history is available, and the entirety of each reviewer's involvement in the project can be resolved to one profile. Additionally, a more accurate calculation of the subsystem and package level expertise can be achieved if the model developers use projects where the ground truth software architecture is known. To implement ELF in a real workflow, model developers need to (1) extract all code reviews along with pull request and commit histories from the version control systems of target projects, (2) pre-calculate the ownership ratios for each example, (3) train their code review comment generation model with the ELF methodology, (4) deploy the static model in their code review platform and (5) repeat steps 1–4 with newer code reviews when performance degradation is observed.

Whilst open-source code review comment generation models are still in the early stages of development, our findings highlight the importance of attending to examples involving expert feedback during model training. As such, practitioners should consider integrating reviewer experience into future model development as it can help optimise the usefulness of such tools for the developers they assist.

Researchers. Whilst our preliminary results show promising signs for integrating reviewer experience into the training process of code review comment generation models, there are various areas for improvement. First, the accuracy of subsystem and package level experience estimation is dependent on the ability to approximate ground truth software architectures. However, manual recovery of these architectures is unfeasible given the number of projects needed for building state-of-the-art models; therefore, our approach relied on a heuristic-based approximation that can be sub-optimal. Future researchers should explore the potential integration of methods in automatic software architecture recovery, such as those that rely on data mining [16], information retrieval [63] or clustering [2, 56, 95]. This can facilitate more accurate experience estimation for fine-grained ownership ratios, thus improving the reliability of the ELF method. Second, our ability to holistically evaluate the techniques on the entire test set is restricted by the absence of reliable automatic evaluation methods. As a result, our study relied on costly manual evaluation, which only scales to small samples. Future researchers should investigate reliable automatic methods to evaluate natural language code reviews in terms of semantic equivalence, applicability, informativeness measures and comment categories. This can facilitate rapid development, e.g., search optimal parameter configurations or loss function designs. Additionally, the domain knowledge of experienced reviewers can often be project-specific and dependent on contextual information

beyond the input code hunk. As a result, the limited contextual information presents a restrictive upper bound on the effectiveness of the ELF method. Future researchers should investigate ways to retrieve and incorporate relevant project information discussed by the code review comment.

Whilst this study integrates the notion of reviewer experience into the fine-tuning process of code review comment generation models, the idea can be applied to any AI for software engineering task involving human-created artefacts. The only condition is that developer characteristics must induce a meaningful difference in the artefact of concern. Additionally, the use of ownership ratios is not restricted to weighting loss functions and can be incorporated into any technique that uses quantifiable measures, e.g., weighting the ranking in retrieval systems [43] or weighted data sampling [11, 59]. In summary, our study sheds light on the utility in considering the characteristics of developers behind the targeted artefacts, which is a perspective that should be considered more generally by all researchers working on AI for software engineering.

9 Threats to Validity

We now discuss the threats to the validity of our study.

Internal Validity. To ensure that our representation of CodeReviewer is faithful to the original paper [47], we utilised their exact replication package, pre-trained checkpoint, hyper-parameters and training setup. The only varying factors are the filtered dataset used for fine-tuning and the experience-aware techniques that we introduce. Ownership values may be underestimated when records of reviews or commits are lost, users are unsearchable, users use multiple accounts, and when projects are rebased or deleted. As BLEU-4 is not an adequate measure for comprehensively evaluating the correctness of the automated code review models, we not only employ manual evaluation to capture semantic equivalence with the ground truth but we also assess the applicability of the comments irrespective of the ground truth. To capture the informativeness of generated reviews, we conduct manual evaluation in terms of both feedback type and presence of explanation. To capture the change in behaviour in terms of the subject of generated comments, we conduct manual evaluation on the review category type. Since manual evaluation is prone to subjectivity and bias, the two annotators conducted the manual evaluations independently with the sources of the generated reviews masked. All conflicts were resolved together until a satisfactory inter-rater agreement level was reached, resulting in a refined guideline that was used to complete the rest of the annotations. Two additional reviewers then independently checked the final results for consistency. To manage the scale of manual annotation, we sampled at a 95% confidence level with a 10% margin of error, rather than a more precise 5% margin of error. Nevertheless, we reported statistically significant results under the one-tailed two proportion Z-test. All research outputs are included in the replication package for transparency.[4]

External Validity. Our experiments on model training focused on 519 of the most starred projects on GitHub, with more than 2,500 pull requests. Thus, the behaviours elicited from our ELF technique may not generalise to smaller scale software projects or to developers in other code review environments, e.g., Gerrit or closed-source development. Additionally, the dataset consists of only inline code review comments, which dictates the nature of the discussion. As such, these findings may not generalise to other types of code review comments, e.g., commit level or pull request level.

10 Conclusion

The field of code review comment generation has demonstrated the potential of deep learning-based language models in automating the cognitively loaded task of code reviewing. Whilst past studies have focused on technical improvements using techniques derived from the field of machine

[4] https://zenodo.org/records/15400484.

learning, our study explores the potential of leveraging the software engineering concept of reviewer experience to elicit higher quality code reviews from automated code review models. Our proposed ELF method re-weights the training data using traditional ownership metrics that reflect a reviewer's authoring and reviewing experiences at the repository, subsystem and package level. Through both quantitative and qualitative evaluation, our results show that certain ELF models can surpass all past methods in terms of both accuracy (RQ1) and informativeness (RQ2), whilst also identifying more functional and evolvability issues (RQ3) in the process. In particular, we found that considering both authoring and reviewing experiences separately at the package level was the most beneficial; however, the uniqueness of the comments elicited from both the repository and subsystem level view also demonstrates that different granularities are highly complementary. We hope that our findings can inspire future work in review comment generation to also consider integrating established software engineering concepts and theories into the design of automated code review models.

References

[1] Miltiadis Allamanis, Earl T. Barr, Premkumar T. Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys* 51, 4 (2018), 81:1–81:37. DOI: https://doi.org/10.1145/3212695

[2] Nicolas Anquetil and Timothy C. Lethbridge. 2009. Ten years later, experiments with clustering as a software remodularization method. In *WCRE*. Andy Zaidman, Giuliano Antoniol, and Stéphane Ducasse (Eds.), IEEE, New York, NY, 7. DOI: https://doi.org/10.1109/WCRE.2009.59

[3] Guilherme Avelino, Leonardo Teixeira Passos, André C. Hora, and Marco Túlio Valente. 2016. A novel approach for estimating truck factors. In *ICPC*. IEEE, New York, NY, 1–10. DOI: https://doi.org/10.1109/ICPC.2016.7503718

[4] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *ICSE*. IEEE, New York, NY, 712–721. DOI: https://doi.org/10.1109/ICSE.2013.6606617

[5] Sebastian Baltes, Christoph Treude, and Martin P. Robillard. 2022. Contextual documentation referencing on stack overflow. *IEEE Transactions on Software Engineering* 48, 1 (2022), 135–149. DOI: https://doi.org/10.1109/TSE.2020.2981898

[6] Rajendra K. Bandi, Vijay K. Vaishnavi, and Daniel E. Turk. 2003. Predicting maintenance performance using object-oriented design complexity metrics. *IEEE Transactions on Software Engineering* 29, 1 (2003), 77–87. DOI: https://doi.org/10.1109/TSE.2003.1166590

[7] Tobias Baum, Olga Liskin, Kai Niklas, and Kurt Schneider. 2016. Factors influencing code review processes in industry. In *ESEC/FSE*. ACM, New York, NY, 85–96. DOI: https://doi.org/10.1145/2950290.2950323

[8] Moritz Beller, Alberto Bacchelli, Andy Zaidman, and Elmar Juergens. 2014. Modern code reviews in open-source projects: Which problems do they fix? In *MSR*. ACM, New York, NY, 202–211. DOI: https://doi.org/10.1145/2597073.2597082

[9] Christian Bird, Nachiappan Nagappan, Brendan Murphy, Harald C. Gall, and Premkumar T. Devanbu. 2011. Don't touch my code!: Examining the effects of ownership on software quality. In *ESEC/FSE*. ACM, New York, NY, 4–14. DOI: https://doi.org/10.1145/2025113.2025119

[10] Amiangshu Bosu, Michaela Greiler, and Christian Bird. 2015. Characteristics of useful code reviews: An empirical study at Microsoft. In *MSR*. IEEE, New York, NY, 146–156. DOI: https://doi.org/10.1109/MSR.2015.21

[11] Haw-Shiuan Chang, Erik Learned-Miller, and Andrew McCallum. 2017. Active bias: training more accurate neural networks by emphasizing high variance samples. In *NeurIPS*. Curran Associates Inc., Red Hook, NY, 1003–1013. Retrieved from https://papers.nips.cc/paper_files/paper/2017/hash/2f37d10131f2a483a8dd005b3d14b0d9-Abstract.html

[12] Tse-Hsun Chen, Meiyappan Nagappan, Emad Shihab, and Ahmed E. Hassan. 2014. An empirical study of dormant bugs. In *MSR*. ACM, New York, NY, 82–91. DOI: https://doi.org/10.1145/2597073.2597108

[13] Moataz Chouchen, Ali Ouni, Raula Gaikovina Kula, Dong Wang, Patanamon Thongtanunam, Mohamed Wiem Mkaouer, and Kenichi Matsumoto. 2021. Anti-patterns in modern code review: Symptoms and prevalence. In *SANER*. IEEE, New York, NY, 531–535. DOI: https://doi.org/10.1109/SANER50967.2021.00060

[14] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.

[15] Jacek Czerwonka, Michaela Greiler, and Jack Tilford. 2015. Code reviews do not find bugs. How the current code review best practice slows Us Down. In *ICSE*, Vol. 2, IEEE, New York, NY, 27–28. DOI: https://doi.org/10.1109/ICSE.2015.131

[16] Carlos Montes de Oca and Doris L. Carver. 1998. Identification of data cohesive subsystems using data mining techniques. In *ICSM*. IEEE, New York, NY, 16–23. DOI: https://doi.org/10.1109/ICSM.1998.738485
[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*. ACL, Philadelphia, PA, 4171–4186. DOI: https://doi.org/10.18653/V1/N19-1423
[18] Edson Dias, Paulo Meirelles, Fernando Castor, Igor Steinmacher, Igor Wiese, and Gustavo Pinto. 2021. What makes a great maintainer of open source projects? In *ICSE*. IEEE, New York, NY, 982–994. DOI: https://doi.org/10.1109/ICSE43902.2021.00093
[19] Yafen Dong, Xiaohong Shen, Zhe Jiang, and Haiyan Wang. 2021. Recognition of imbalanced underwater acoustic datasets with exponentially weighted cross-entropy loss. *Applied Acoustics* 174 (2021), 107740. DOI: https://doi.org/10.1016/J.APACOUST.2020.107740
[20] Felipe Ebert, Fernando Castor, Nicole Novielli, and Alexander Serebrenik. 2018. Communicative intention in code review questions. In *ICSME*. IEEE, New York, NY, 519–523. DOI: https://doi.org/10.1109/ICSME.2018.00061
[21] Joseph L. Fleiss, Bruce Levin, and Myunghee Cho Paik. 2013. *Statistical Methods for Rates and Proportions*. John Wiley & Sons, Hoboken, NJ.
[22] Thomas Fritz, Gail C. Murphy, and Emily Hill. 2007. Does a programmer's activity indicate knowledge of code? In *ESEC/FSE*. ACM, New York, NY, 341–350. DOI: https://doi.org/10.1145/1287624.1287673
[23] Alexander Frömmgen, Jacob Austin, Peter Choy, Nimesh Ghelani, Lera Kharatyan, Gabriela Surita, Elena Khrapko, Pascal Lamblin, Pierre-Antoine Manzagol, Marcus Revaj, et al. 2024. Resolving code review comments with machine learning. In *ICSE-SEIP*. IEEE, New York, NY, 204–215. DOI: https://doi.org/10.1145/3639477.3639746
[24] Michael Fu, Van Nguyen, Chakkrit Tantithamthavorn, Dinh Phung, and Trung Le. 2024. Vision transformer inspired automated vulnerability repair. *ACM Transactions on Software Engineering and Methodology* 33, 3, Article 78 (Mar. 2024), 29 pages. DOI: https://doi.org/10.1145/3632746
[25] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A transformer-based line-level vulnerability prediction. In *MSR*. ACM, New York, NY, 608–620. DOI: https://doi.org/10.1145/3524842.3528452
[26] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *ESEC/FSE*. ACM, New York, NY, 935–947. DOI: https://doi.org/10.1145/3540250.3549098
[27] Akalanka Galappaththi, Sarah Nadi, and Christoph Treude. 2022. Does this apply to me? An empirical study of technical context in stack overflow. In *MSR*. ACM, New York, NY, 23–34. DOI: https://doi.org/10.1145/3524842.3528435
[28] Haoyu Gao, Christoph Treude, and Mansooreh Zahedi. 2023. Evaluating transfer learning for simplifying GitHub READMEs. In *ESEC/FSE*. ACM, New York, NY, 1548–1560. DOI: https://doi.org/10.1145/3611643.3616291
[29] Mehdi Golzadeh, Alexandre Decan, and Natarajan Chidambaram. 2022. On the accuracy of bot detection techniques. In *BotSE*. ACM, New York, NY, 1–5. DOI: https://doi.org/10.1145/3528228.3528406
[30] Mehdi Golzadeh, Alexandre Decan, Damien Legay, and Tom Mens. 2021. A ground-truth dataset and classification model for detecting bots in GitHub issue and PR comments. *Journal of Systems and Software* 175 (2021), 110911. DOI: https://doi.org/10.1016/J.JSS.2021.110911
[31] Pavlína Wurzel Gonçalves, Enrico Fregnan, Tobias Baum, Kurt Schneider, and Alberto Bacchelli. 2022. Do explicit review strategies improve code review performance? Towards understanding the role of cognitive load. *Empirical Software Engineering* 27, 4 (2022), 99. DOI: https://doi.org/10.1007/S10664-022-10123-8
[32] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *ICSE*. IEEE, New York, NY, 1–13. DOI: https://doi.org/10.1145/3597503.3623306
[33] Masum Hasan, Anindya Iqbal, Mohammad Rafid Ul Islam, A. J. M. Imtiajur Rahman, and Amiangshu Bosu. 2021. Using a balanced scorecard to identify opportunities to improve code review effectiveness: An industrial experience report. *Empirical Software Engineering* 26 (2021), 1–34. DOI: https://doi.org/10.1007/S10664-021-10038-W
[34] Hideaki Hata, Osamu Mizuno, and Tohru Kikuno. 2012. Bug prediction based on fine-grained module histories. In *ICSE*. IEEE, New York, NY, 200–210. DOI: https://doi.org/10.1109/ICSE.2012.6227193
[35] Vincent J. Hellendoorn, Jason Tsay, Manisha Mukherjee, and Martin Hirzel. 2021. Towards automating code review at scale. In *ICSE*. IEEE, New York, NY, 1479–1482. DOI: https://doi.org/10.1145/3468264.3473134
[36] Abram Hindle, Earl T. Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Communications of the ACM* 59, 5 (2016), 122–131. DOI: https://doi.org/10.1145/2902362
[37] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-aware neural machine translation for automatic program repair. In *ICSE*. IEEE, New York, NY, 1161–1173. DOI: https://doi.org/10.1109/ICSE43902.2021.00107
[38] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Inferfix: End-to-end program repair with llms. In *ESEC/FSE*. ACM, New York, NY, 1646–1656. DOI: https://doi.org/10.1145/3611643.3613892

[39] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2016. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering* 21 (2016), 2035–2071. DOI: https://doi.org/10.1007/S10664-015-9393-5

[40] Oleksii Kononenko, Olga Baysal, and Michael W. Godfrey. 2016. Code review quality: How developers see it. In *ICSE*. IEEE, New York, NY, 1028–1038. DOI: https://doi.org/10.1145/2884781.2884840

[41] Oleksii Kononenko, Olga Baysal, Latifa Guerrouj, Yaxin Cao, and Michael W. Godfrey. 2015. Investigating code review quality: Do people and participation matter? In *ICSME*. IEEE, New York, NY, 111–120. DOI: https://doi.org/10.1109/ICSM.2015.7332457

[42] Philippe B. Kruchten. 1995. The 4+ 1 view model of architecture. *IEEE Software* 12, 6 (1995), 42–50. DOI: https://doi.org/10.1109/52.469759

[43] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-Tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*. Curran Associates Inc., Red Hook, NY, Article 793, 16 pages. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[44] Jia Li, Ge Li, Zhuo Li, Zhi Jin, Xing Hu, Kechi Zhang, and Zhiyi Fu. 2023. Codeeditor: Learning to edit source code with pre-trained models. *ACM Transactions on Software Engineering and Methodology* 32, 6 (2023), 1–22. DOI: https://doi.org/10.1145/3597207

[45] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. AUGER: Automatically generating review comments with pre-training models. In *ESEC/FSE*. ACM, New York, NY, 1009–1021. DOI: https://doi.org/10.1145/3540250.3549099

[46] Yi Li, Yiduo Yu, Yiwen Zou, Tianqi Xiang, and Xiaomeng Li. 2022. Online easy example mining for weakly-supervised gland segmentation from histology images. In *MICCAI*. Springer, New York, NY, 578–587. DOI: https://doi.org/10.1007/978-3-031-16440-8_55

[47] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. 2022. Automating code review activities by large-scale pre-training. In *ESEC/FSE*. ACM, New York, NY, 1035–1047. DOI: https://doi.org/10.1145/3540250.3549081

[48] Bo Lin, Shangwen Wang, Zhongxin Liu, Yepang Liu, Xin Xia, and Xiaoguang Mao. 2023. Cct5: A code-change-oriented pre-trained model. In *ESEC/FSE*. ACM, New York, NY, 1509–1521. DOI: https://doi.org/10.1145/3611643.3616339

[49] Hong Yi Lin and Patanamon Thongtanunam. 2023. Towards automated code reviews: Does learning code structure help? In *SANER*. IEEE, New York, NY, 703–707. DOI: https://doi.org/10.1109/SANER56733.2023.00075

[50] Hong Yi Lin, Patanamon Thongtanunam, Christoph Treude, and Wachiraphan Charoenwet. 2024. Improving automated code reviews: Learning from experience. In *MSR*. IEEE, New York, NY, 278–283. DOI: https://doi.org/10.1145/3643991.3644910

[51] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-Reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *ISSRE*. IEEE, New York, NY, 647–658. DOI: https://doi.org/10.1109/ISSRE59848.2023.00026

[52] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS Datasets*. Curran Associates Inc., Red Hook, NY. Retrieved from https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html

[53] Laura MacLeod, Michaela Greiler, Margaret-Anne Storey, Christian Bird, and Jacek Czerwonka. 2018. Code reviewing in the trenches: Challenges and best practices. *IEEE Software* 35, 4 (2018), 34–42. DOI: https://doi.org/10.1109/MS.2017.265100500

[54] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The impact of code review coverage and code review participation on software quality: A case study of the qt, VTK, and ITK projects. In *MSR*. ACM, New York, NY, 192–201. DOI: https://doi.org/10.1145/2597073.2597076

[55] Xiaozhu Meng, Barton P. Miller, William R. Williams, and Andrew R. Bernat. 2013. Mining software repositories for accurate authorship. In *ICSME*. IEEE, New York, NY, 250–259. DOI: https://doi.org/10.1109/ICSM.2013.36

[56] Brian S. Mitchell and Spiros Mancoridis. 2006. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering* 32, 3 (2006), 193–208. DOI: https://doi.org/10.1109/TSE.2006.31

[57] Gail C. Murphy, David Notkin, and Kevin Sullivan. 1995. Software reflexion models: Bridging the gap between source and high-level models. In *ESEC/FSE*. ACM, New York, NY, 18–28. DOI: https://doi.org/10.1145/222124.222136

[58] Mika V. Mäntylä and Casper Lassenius. 2009. What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering* 35, 3 (2009), 430–448. DOI: https://doi.org/10.1109/TSE.2008.71

[59] Deanna Needell, Nathan Srebro, and Rachel Ward. 2014. Stochastic gradient descent, weighted sampling, and the randomized kaczmarz algorithm. In *NeurIPS*. MIT Press, Cambridge, MA, 1017–1025. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2014/hash/b3310bba2be31e673a7ded3386994599-Abstract.html

[60] Paul W. Oman and Curtis R. Cook. 1990. Typographic style is more than cosmetic. *Communications of the ACM* 33, 5 (May 1990), 506–520. DOI: https://doi.org/10.1145/78607.78611
[61] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: A method for automatic evaluation of machine translation. In *ACL*. ACL, Philadelphia, PA, 311–318. DOI: https://doi.org/10.3115/1073083.1073135
[62] Chanathip Pornprasit, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, and Chunyang Chen. 2023. D-act: Towards diff-aware code transformation for code review under a time-wise evaluation. In *SANER*. IEEE, New York, NY, 296–307. DOI: https://doi.org/10.1109/SANER56733.2023.00036
[63] Denys Poshyvanyk, Malcom Gethers, and Andrian Marcus. 2012. Concept location using formal concept analysis and information retrieval. *ACM Transactions on Software Engineering and Methodology* 21, 4 (2012), 23:1–23:34. DOI: https://doi.org/10.1145/2377656.2377660
[64] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. *Improving Language Understanding by Generative Pre-Training*. Technical Report. OpenAI.
[65] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
[66] Foyzur Rahman and Premkumar Devanbu. 2011. Ownership, experience and defects: A fine-grained study of authorship. In *ICSE*. IEEE, New York, NY, 491–500. DOI: https://doi.org/10.1145/1985793.1985860
[67] Mohammad Masudur Rahman, Chanchal K. Roy, and Raula G. Kula. 2017. Predicting usefulness of code review comments using textual features and developer experience. In *MSR*. IEEE, New York, NY, 215–226. DOI: https://doi.org/10.1109/MSR.2017.17
[68] Shadikur Rahman, Umme Ayman Koana, and Maleknaz Nayebi. 2022. Example driven code review explanation. In *ESEM*. ACM, New York, NY, 307–312. DOI: https://doi.org/10.1145/3544902.3546639
[69] Baishakhi Ray, Vincent Hellendoorn, Saheel Godhane, Zhaopeng Tu, Alberto Bacchelli, and Premkumar Devanbu. 2016. On the "naturalness" of buggy code. In *ICSE*. IEEE, New York, NY, 428–439. DOI: https://doi.org/10.1145/2884781.2884848
[70] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *ESEC/FSE*. ACM, New York, NY, 202–212. DOI: https://doi.org/10.1145/2491411.2491444
[71] H. D. Rombach. 1987. A controlled experiment on the impact of software structure on maintainability. *IEEE Transactions on Software Engineering* SE-13, 3 (1987), 344–354. DOI: https://doi.org/10.1109/TSE.1987.233165
[72] Oussama Ben Sghaier and Houari A. Sahraoui. 2024. Improving the learning of code review successive tasks with Cross-Task knowledge distillation. In *ESEC/FSE*, Vol. 1, ACM, New York, NY, 1086–1106. DOI: https://doi.org/10.1145/3643775
[73] Davide Spadini, Maurício Aniche, and Alberto Bacchelli. 2018. PyDriller: Python framework for mining software repositories. In *ESEC/FSE*. ACM, New York, NY, 908–911. DOI: https://doi.org/10.1145/3236024.3264598
[74] M.-A. D. Storey, F. David Fracchia, and Hausi A. Müller. 1999. Cognitive design elements to support the construction of a mental model during software exploration. *Journal of Systems and Software* 44, 3 (1999), 171–185. DOI: https://doi.org/10.1016/S0164-1212(98)10055-9
[75] Patanamon Thongtanunam, Shane McIntosh, Ahmed E. Hassan, and Hajimu Iida. 2015. Investigating code review practices in defective files: An empirical study of the Qt system. In *MSR*. ACM, New York, NY, 168–179. DOI: https://doi.org/10.1109/MSR.2015.23
[76] Patanamon Thongtanunam, Shane McIntosh, Ahmed E. Hassan, and Hajimu Iida. 2016. Revisiting code ownership and its relationship with software quality in the scope of modern code review. In *ICSE*. IEEE, New York, NY, 1039–1050. DOI: https://doi.org/10.1145/2884781.2884852
[77] Patanamon Thongtanunam, Chanathip Pornprasit, and Chakkrit Tantithamthavorn. 2022. Autotransform: Automated code transformation to support modern code review process. In *ICSE*. IEEE, New York, NY, 237–248. DOI: https://doi.org/10.1145/3510003.3510067
[78] Patanamon Thongtanunam and Chakkrit Tantithamthavorn. 2024. Code ownership: The principles, differences, and their associations with software quality. In *ISSRE*. IEEE, New York, NY.
[79] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-Ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for modern code review. In *SANER*. IEEE, New York, NY, 141–150. DOI: https://doi.org/10.1109/SANER.2015.7081824
[80] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from https://arxiv.org/abs/2302.13971
[81] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *ICSE*. IEEE, New York, NY, 25–36. DOI: https://doi.org/10.1109/ICSE.2019.00021

[82] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 1–29. DOI: https://doi.org/10.1145/3340544

[83] Rosalia Tufano, Ozren Dabić, Antonio Mastropaolo, Matteo Ciniselli, and Gabriele Bavota. 2024. Code review automation: Strengths and weaknesses of the state of the art. *IEEE Transactions on Software Engineering* 50, 2 (2024), 338–353. DOI: https://doi.org/10.1109/TSE.2023.3348172

[84] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *ICSE*. IEEE, New York, NY, 2291–2302. DOI: https://doi.org/10.1145/3510003.3510621

[85] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards automating code review activities. In *ICSE*. IEEE, New York, NY, 163–174. DOI: https://doi.org/10.1109/ICSE43902.2021.00027

[86] Asif Kamal Turzo and Amiangshu Bosu. 2024. What makes a code review useful to opendev developers? An empirical investigation. *Empirical Software Engineering* 29, 1 (2024), 6. DOI: https://doi.org/10.1007/S10664-023-10411-X

[87] Asif Kamal Turzo, Fahim Faysal, Ovi Poddar, Jaydeb Sarker, Anindya Iqbal, and Amiangshu Bosu. 2023. Towards automated classification of code review feedback to support analytics. In *ESEM*. IEEE, New York, NY, 1–12. DOI: https://doi.org/10.1109/ESEM56168.2023.10304851

[88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*. Curran Associates Inc., Red Hook, NY, 5998–6008. Retrieved from https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[89] Manushree Vijayvergiya, Małgorzata Salawa, Ivan Budiselić, Dan Zheng, Pascal Lamblin, Marko Ivanković, Juanjo Carin, Mateusz Lewko, Jovan Andonov, Goran Petrović, et al. 2024. AI-Assisted assessment of coding practices in modern code review. In *AIWARE*. ACM, New York, NY, 85–93. DOI: https://doi.org/10.1145/3664646.3665664

[90] Valery Vishnevskiy, Richard Rau, and Orcun Goksel. 2019. Deep variational networks with exponential weighting for learning computed tomography. In *MICCAI*. Springer, New York, NY, 310–318. DOI: https://doi.org/10.1007/978-3-030-32226-7_35

[91] Kai Wang, Yizhou Peng, Hao Huang, Ying Hu, and Sheng Li. 2022. Mining hard samples locally and globally for improved speech separation. In *ICASSP*. IEEE, New York, NY, 6037–6041. DOI: https://doi.org/10.1109/ICASSP43922.2022.9747797

[92] Xinshao Wang, Yang Hua, Elyor Kodirov, Guosheng Hu, and Neil Martin Robertson. 2019. Deep metric learning by online soft mining and class-aware attention. In *AAAI*. AAAI Press, Washington D.C, 5361–5368. DOI: https://doi.org/10.1609/AAAI.V33I01.33015361

[93] Jiwei Wei, Yang Yang, Xing Xu, Jingkuan Song, Guoqing Wang, and Heng Tao Shen. 2023. Less is better: Exponential loss for cross-modal matching. *IEEE Transactions on Circuits and Systems for Video Technology* 33, 9 (2023), 5271–5280. DOI: https://doi.org/10.1109/TCSVT.2023.3249754

[94] Mairieli Wessel, Bruno Mendes de Souza, Igor Steinmacher, Igor S. Wiese, Ivanilton Polato, Ana Paula Chaves, and Marco A. Gerosa. 2018. The power of bots: Characterizing and understanding bots in OSS projects. In *CSCW*, Vol. 2, ACM, New York, NY, Article 182, 19 pages. DOI: https://doi.org/10.1145/3274451

[95] T. A. Wiggerts. 1997. Using Clustering Algorithms in Legacy Systems Remodularization. In *WCRE*. Ira D. Baxter, Alex Quilici, and Chris Verhoef (Eds.), IEEE, New York, NY, 33–43. DOI: https://doi.org/10.1109/WCRE.1997.624574

[96] Lanxin Yang, Jinwei Xu, Yifan Zhang, He Zhang, and Alberto Bacchelli. 2023. EvaCRC: Evaluating code review comments. In *ESEC/FSE*. ACM, New York, NY, 275–287. DOI: https://doi.org/10.1145/3611643.3616245

[97] Motahareh Bahrami Zanjani, Huzefa Kagdi, and Christian Bird. 2016. Automatically recommending peer reviewers in modern code review. *IEEE Transactions on Software Engineering* 42, 6 (2016), 530–543. DOI: https://doi.org/10.1109/TSE.2015.2500238

[98] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *ISSTA*. ACM, New York, NY, 39–51. DOI: https://doi.org/10.1145/3533767.3534390

[99] Beiqi Zhang, Liming Fu, Peng Liang, Jiaxin Yu, and Chong Wang. 2024. Demystifying code snippets in code reviews: A study of the OpenStack and Qt communities and a practitioner survey. *Empirical Software Engineering* 29, 4 (2024), 78. DOI: https://doi.org/10.1007/s10664-024-10484-2

[100] Xin Zhou, Kisub Kim, Bowen Xu, Donggyun Han, and David Lo. 2024. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources. In *ICSE*. IEEE, New York, NY, Article 88, 13 pages. DOI: https://doi.org/10.1145/3597503.3639222

Received 17 September 2024; revised 19 May 2025; accepted 10 August 2025